Mitigating Inter-Process Communication Overhead in seL4-Based Systems: An Asynchronous, Multiclient File Server

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Luke Rule

Supervised by James Garside

Department of Computer Science

Abstract

TODO

Contents

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Aims and Objectives 2

1.3 Evaluation Strategy 3

1.4 Key Findings 4

1.5 Dissertation Structure 4

2 Background **5**

2.1 Operating System Kernels 5

2.1.1 Definition and Characteristics 5

2.1.2 Microkernels Versus Monolithic Kernels 6

2.2 seL4 Microkernel 7

2.2.1 Overview and Characteristics 7

2.2.2 Design 8

2.2.3 Use Cases and Applications 9

2.2.4 Microkit Framework for seL4 9

2.3 Client-Server Communication 10

2.3.1 Synchronous Model 10

2.3.2 Asynchronous Model 11

2.4 File Servers in Operating Systems 12

2.4.1 Role and Functionality 12

2.4.2 Abstractions 12

2.4.3 Storage and Organisation 13

File Layout 13

Allocation Strategies 13

Metadata Organisation 15

Directory Structure 17

2.4.4 Client Management 18

2.5 Related Work 18

3 Design **19**
3.1 System Requirements and Constraints . . . 19
3.2 IPC Design and File Server Interface . . . 20
3.2.1 Synchronous Model *(Baseline)* . . . 20
3.2.2 Asynchronous Model *(Proposed IPC Mitigation Strategy)* . . . 22
3.3 File Server . . . 24
3.3.1 Architectural Overview . . . 24
3.3.2 Component Design and Interactions . . . 26
Blocks . . . 26
inodes . . . 27
Directories . . . 28
File Descriptors . . . 29
3.3.3 Request Processing Flow . . . 30
3.3.4 Multiclient Support . . . 31

4 Implementation **32**
4.1 Development Environment and Tools . . . 32
4.2 Microkit Framework Setup . . . 32
4.3 Interface . . . 34
4.3.1 Synchronous Server Model . . . 34
Client-Server Communication . . . 34
API . . . 35
4.3.2 Asynchronous Server Model . . . 36
Client-Server Communication . . . 36
API . . . 40
4.4 File Server . . . 41
4.4.1 Data Structures and Access Mechanisms . . . 41
Blocks . . . 41
inodes . . . 41
Directories . . . 43
File Descriptors . . . 44
4.4.2 Operations . . . 45
Namespace Mutation . . . 45
Runtime State Management . . . 47
Data Movement . . . 48
Metadata Access and Modification . . . 49
4.5 Error Handling . . . 50

5 Evaluation **51**
5.1 Validation . . . 51
5.1.1 Unit Tests . . . 51
5.1.2 Integration Tests . . . 52
Single Client . . . 52
Multiple Clients . . . 52
5.2 Performance Evaluation . . . 53
5.2.1 Goals . . . 53
5.2.2 Metrics . . . 53
5.2.3 Experimental Setup . . . 53
5.2.4 Benchmark Design . . . 54
5.2.5 Results and Analysis . . . 55
Single-Client Per-Operation Benchmark . . . 55
Realistic Multiclient Workload . . . 56

6 Conclusion **59**
6.1 Contributions . . . 59
6.1.1 System Design and Implementation . . . 59
6.1.2 IPC Overhead Mitigation Insights . . . 60
6.2 Drawbacks and Limitations . . . 61
6.2.1 File System Limitations . . . 61
6.2.2 Benchmarking Constraints . . . 61
6.2.3 Lack of Persistent Storage . . . 61
6.2.4 Single-Threaded Execution . . . 62
6.2.5 Dependency on Workload Characteristics . . . 62
6.2.6 Additional Client Burden . . . 62
6.3 Future Work . . . 63

Bibliography **64**

Word Count: 15,000

List of Figures

1.1 Comparison of tightly coupled and loosely coupled system structures. 1

2.1 View of kernel interaction within an operating system. 5
2.2 Comparison of microkernel and monolithic kernel architectures. Adapted from source: [1] 6
2.3 Comparison of ACLs and capability based access control models. Adapted from source: [2] 8
2.4 Build-time and run-time structures for a Microkit-based system. Adapted from source: [3] 9
2.5 Communication channels between PDs. Adapted from source: [3] 10
2.6 Comparison of synchronous and asynchronous communication models. 11
2.7 Multiclient file server structure. 12
2.8 Example of contiguous block allocation. Source: [4]. 14
2.9 Example of linked block allocation. Source: [4]. 14
2.10 Example of indexed block allocation. Source: [4]. 15
2.11 Example of FAT based file management. Source: [4]. 16
2.12 Example of inode based file management with indirect block indexing. Source: [4]. 16
2.13 Example of a hierarchical directory structure. Source: [4]. 17

3.1 Synchronous multiclient-server interaction. 21
3.2 Asynchronous multiclient-server interaction. 23
3.3 Architectural overview of the file server. 25
3.4 Block allocation table structure. 26
3.5 inode and block reference structure. 27
3.6 Directory structure via inodes and blocks. 28
3.7 File descriptor structure. 29

4.1 Asynchronous client-server communication via shared queues and buffers. 36
4.2 Buffer management in asynchronous communication. 37
4.3 Path-based lookup through directory traversal. 43

5.1 Average time per operation by batch size (Read and Write) 55
5.2 Gradient of average time per operation by batch size (Read and Write) 56
5.3 Total execution time per client by number of clients (Synchronous vs Asynchronous) . . . 57
5.4 Client completion time variability (min - max) by number of clients 58

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on presentation of Theses

Acknowledgements

I would like to thank...

Abbreviations

diff tables FD! TODO

1 Introduction

1.1 Context and Motivation

Modern software systems are larger, more complex and more interconnected than ever before [**?**], placing significant demands upon the operating systems which underpin them. Contemporary mainstream operating systems, such as Windows, Linux and macOS [1] are largely based upon monolithic kernel designs[**?**], where the majority of their functionality resides in a single, privileged address space. This key characteristic enables high performance[**?**], yet it comes at the cost of scalability, security and robustness[**?**], as it requires a large Trusted Computing Base (TCB) where a single fault can compromise the entire system[**?**].

Simultaneously, modern software and hardware architectures increasingly employ modularity, isolation and concurrency[**?**]. Distributed systems, cloud-native microservices and data centre-scale applications all decompose functionality into loosely-coupled components[**?**], as illustrated in Figure 1.1, and most current hardware performance scaling is driven by multi-core and many-core parallelism[**?**]. These trends place pressure on operating system design to provide stronger security guarantees and better support for concurrent, component-based execution.

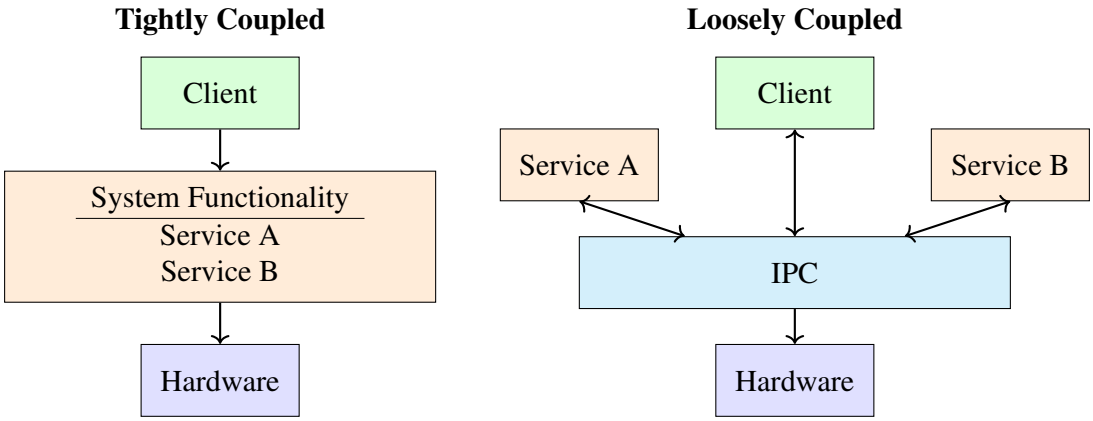


Figure 1.1: Comparison of tightly coupled and loosely coupled system structures.

[1] While macOS is often described as a hybrid kernel[**?**], it retains many monolithic characteristics to such an extent that it loses many of the benefits typically associated with pure microkernel design[**?**].

Microkernel design naturally aligns with these requirements[**?**]. By minimising kernel functionality and moving most non-essential services, such as device drivers and file systems, into user space, microkernels reduce the TCB significantly and enforce separation between components. However, this modularity requires costly Inter-Process Communication (IPC) across protected domains.

The L4 family of microkernels[**?**] addressed this bottleneck by substantially reducing IPC costs through aggressive kernel minimisation and IPC fastpath design[**?**]; yet, despite these advances, IPC overhead remains a fundamental constraint in microkernel-based systems and a barrier to their wider adoption[**?**]. However, by combining these performance improvements with strong security guarantees through formal verification[**?**], the seL4 microkernel represents a significant step towards making microkernel-based designs competitive for real-world deployment, particularly in security and safety-critical domains[**?**], although, performance constraints remain a limiting factor, especially for high-frequency IPC workloads.

One of the most frequently accessed operating system services is the file server[**?**], which acts as the primary interface for persistent storage and data exchange. File system operations are both latency-sensitive and IPC-intensive[**?**], making them critical to overall system performance in microkernel environments. In synchronous architectures even simple file operations may involve multiple IPC operations between the client and the file server, amplifying the impact of communication overhead.

This report presents the design and implementation of an asynchronous, multiclient file server for the seL4 microkernel[**?**], validates its correctness and usability, and evaluates its effectiveness in mitigating IPC overhead. The proposed architecture decouples request submission from response handling, reducing blocking behaviour, increasing concurrency and enabling the *batching* of operations, where multiple requests may be combined, sent and processed together. These properties have the potential to improve throughput and resource utilisation under both single and multiclient workloads, particularly when deployed on increasingly multicore machines[**?**], where parallel execution can be fully exploited. In addition, beyond serving as an evaluation platform, the implementation constitutes a practical, reusable system component, demonstrating how critical services can be structured within a microkernel environment using the seL4 Microkit's component-based design model.

1.2 Aims and Objectives

Building upon the L4 family's reduction of IPC overhead, and motivated by the increasing importance of concurrency in contemporary systems, this work aims to further mitigate IPC costs by reducing the number of interactions required between components. In doing so, it also seeks to demonstrate the capabilities of the seL4 Microkit framework through the design and implementation of a flexible, user-space file server.

This project has the following objectives:

- **Design and implement**:
  - A viable file system core capable of supporting a wide range of file and directory operations.
  - An efficient storage model based on inode and block abstractions, supporting effective space utilisation and flexible file and directory representation.
  - A scalable, asynchronous interface enabling concurrent request submissions and completion handling from multiple clients, alongside a synchronous interface to facilitate comparison between communication models.
  - A lightweight client-side library providing a simple and usable abstraction over the file server interface.
  - A benchmarking framework to evaluate asynchronous file server performance against a synchronous baseline under realistic workloads.
- **Validate**: the correctness of the file server through comprehensive multi-level testing.
- **Evaluate**: the effectiveness of the asynchronous file server architecture in reducing IPC overhead and improving performance, in terms of latency, throughput, and scalability.

1.3 Evaluation Strategy

To evaluate the proposed implementation, this work considered both the correctness of file system operations and the performance characteristics of asynchronous queue-based submission compared to synchronous, blocking calls.

Correctness was validated at multiple stages of granularity – unit, integration and end-to-end testing – to ensure file operations correctly persisted and returned data across a range of scenarios, including edge and error cases.

For performance evaluation, a set of benchmarks was designed to compare the synchronous and asynchronous interfaces under varying workloads and measure key metrics, including latency, throughput, and IPC frequency. Experiments were conducted across different levels of request batching to assess the trade-off between asynchronous overhead in low-batching scenarios and performance gains achieved through reduced IPC at higher batching levels.

The benchmark workloads were designed to approximate realistic file system usage patterns; however, as the system is novel, custom benchmarks were required. While these may not have captured all characteristics of production environments, they were sufficient to provide indicative and comparative insights into the performance impact of an asynchronous paradigm.

1.4 Key Findings

The evaluation demonstrates that asynchronous communication introduces negligible overhead compared to synchronous execution when no batching is applied, with per-operation latencies remaining effectively equivalent. Crucially, as batching is introduced, significant performance improvements are observed. For single-client workloads, increasing batch sizes reduces per-operation latency by up to 60% for reads and 50% for writes, demonstrating that reducing IPC frequency has a substantial impact on performance.

These improvements extend to more realistic workloads. In a multiclient benchmark simulating typical file system operations with batching opportunities, the asynchronous model achieves an average latency reduction of approximately 35–40% compared to the synchronous baseline. Notably, these experiments were conducted on a single core, as current seL4 deployments do not fully utilise multicore execution; consequently, greater throughput would be expected from asynchronous designs in a multicore setting due to increased concurrency, allowing requests to be issued and responses processed in parallel.

However, all evaluations were performed using an in-memory storage model rather than disk-backed persistence. While this isolates IPC and scheduling overhead, real-world performance gains would be less pronounced when incorporating variable disk I/O latency. Nevertheless, the results demonstrate that batching and non-blocking execution can significantly reduce IPC overhead, particularly in high-frequency workloads.

1.5 Dissertation Structure

The report begins by presenting the background knowledge required to understand the design of the file server and its architectural context. It then introduces the system design, including the overall architecture, interface models, and key design decisions.

Following this, the implementation of the file server and its two interfaces are described, before outlining the validation strategy and performance evaluation methodology, assessing correctness and comparing the performance of the asynchronous and synchronous communication models.

The report then discusses related work, situating the contribution within existing research, and concludes with a summary of findings, limitations and directions for future work.

2 Background

This section provides the necessary background for understanding the design and implementation of the file server. It introduces the key concepts underlying file servers and microkernel systems, with a focus on inter-process communication in seL4 and its impact on system performance. These concepts form the foundation for the designs explored in the following sections.

2.1 Operating System Kernels

2.1.1 Definition and Characteristics

The kernel is the core component of an operating system, executing at the highest privilege level of the processor (typically referred to as supervisor mode[**?**]). It provides the fundamental interface between hardware and software, managing critical system resources, as shown in Figure 2.1.

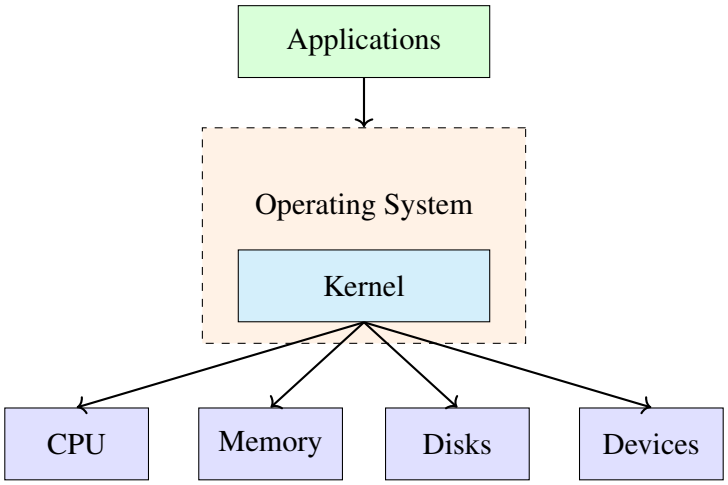


Figure 2.1: View of kernel interaction within an operating system.

The specific set of managed resources varies by system design, but at a minimum the kernel must provide services that cannot be implemented safely or reliably in user space, such as memory management, device management and process or thread scheduling[**?**]. By acting as the sole authority over these resources, the kernel enforces isolation and protection, preventing unintended or malicious interference between processes or to the system as a whole[**?**].

2.1.2 Microkernels Versus Monolithic Kernels

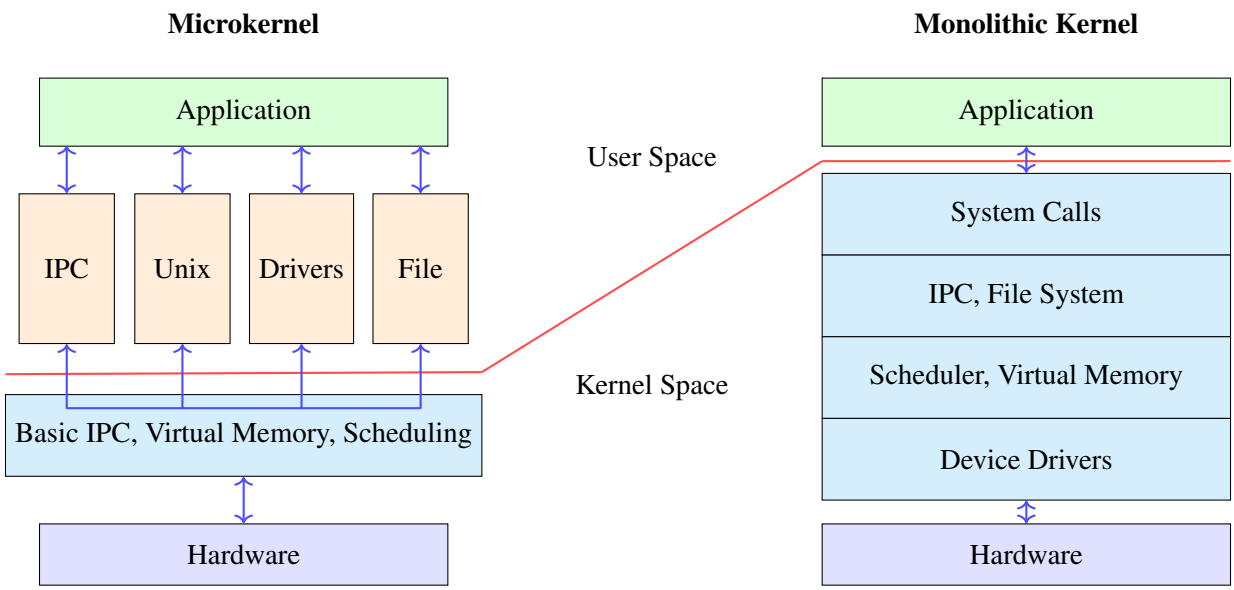


Figure 2.2: Comparison of microkernel and monolithic kernel architectures. Adapted from source: [1]

In addition to the required functionality, monolithic kernels also incorporate other low-level services such as device drivers and file systems directly within the kernel address space, as shown in Figure 2.2. All operating system services can then be accessed via system calls[**?**], which transfer control from user space to kernel space through a restricted privilege level switch[**?**]. Because all these services execute within the same privileged address space, interactions between kernel components can be performed through efficient function calls, with direct access to process data structures. This results in low-overhead service invocation, facilitating high-performance systems[**?**].

From a design perspective, monolithic kernels therefore benefit from simpler service integration – yet, this leads complexity rising factorially as the system grows. The inherent tight coupling of components can make them more difficult to extend, debug, and maintain[**?**], as faults may propagate across sub-system boundaries and be harder to isolate[**?**]. Combined with the larger size of monolithic kernels, this increases the likelihood of defects within the trusted computing base[**?**]. This is significant, as faults within the kernel can have system-wide consequences due to its privileged execution context[**?**].

In contrast, microkernels predominantly restrict kernel functionality to essential mechanisms, delegating higher-level services to user-space components, which must communicate via message passing across their protection boundaries[**?**], as illustrated in Figure 2.2. As a result, invoking a service introduces considerably more overhead when compared to monolithic architectures[**?**]. This primarily arises from context switching and scheduler involvement, when the server resides in a separate process from the client, potentially disrupting processor caches and pipelines[**?**]. However, various optimisations – discussed in Section 2.2 – can significantly reduce these costs.

Additionally, because services are executed in user space and processes reside in separate address spaces, data may need to be copied or explicitly shared between them, introducing further overhead. In some cases, this cost can be mitigated through alternative communication mechanisms. Shared memory[**?**], for example, allows processes to exchange data without copying by mapping a common memory region into multiple processes' address spaces. When combined with techniques such as lock-free data structures[**?**], communication can occur with minimal kernel involvement. These approaches, however, require careful design to ensure correct synchronisation, memory ordering and preservation of isolation guarantees[**?**].

Despite these challenges, the reduced kernel size and strong isolation between components in microkernel architectures enable significantly improved security assurance and fault containment[**?**]. These advantages motivated the design of the L4 family of microkernels, which demonstrated that IPC overhead can be significantly reduced through careful kernel design and optimisation[**?**].

2.2 seL4 Microkernel

2.2.1 Overview and Characteristics

In 1993, the L4 microkernel was developed by Jochen Liedtke in response to the poor performance of earlier microkernel designs[**?**], based on the premise that prioritising efficiency was essential for practical adoption. By recognising IPC as the dominant factor in microkernel performance and optimising these mechanisms accordingly, the L4 design achieved approximately a 20× performance improvement over Mach[**?**], the most widely used microkernel at the time[**?**]. This demonstrated that microkernels could achieve competitive performance, leading to significant further research and the development of multiple L4-derived systems[**?**], including seL4.

seL4 (Security-Enhanced L4) extends this architecture by combining the high-performance IPC with comprehensive formal verification[**?**], becoming the only microkernel to do so[**?**]. Through the complete kernel implementation being proven correct against its specification, seL4 ensures that system calls behave as specified and that key safety properties – such as memory isolation – are upheld. Assuming correct hardware behaviour[**?**], this provides a high degree of assurance that kernel integrity and availability are maintained and that information is not *leaked* between processes, only explicitly shared[**?**].

These guarantees apply only to the kernel itself, however. User-space components are rarely formally verified[**?**] and may still exhibit incorrect behaviour and cause failures at that level. While such faults are isolated and cannot compromise kernel correctness, they may still impact overall system functionality.

This separation of concerns makes seL4 particularly well-suited to exploring user-space approaches to performance optimisation, as the minimal and stable kernel design enables flexibility in higher-level system components, without modifying the verified core.

2.2.2 Design

The L4 specification optimises IPC through a reduced kernel size and efficient message passing[**?**]. Earlier microkernel designs attempted to support richer communication abstractions within the kernel, introducing additional and often unnecessary overhead[**?**]. In contrast, L4 adopts a minimal design, providing only basic, synchronous IPC primitives[**?**]. This approach prioritises low latency in the common case, removing more complex mechanisms – such as asynchronous buffering and batching – from the kernel[**?**]. These may instead be constructed externally when required, utilising the efficiency of the primitives provided without burdening the kernel.

Kernel involvement is also minimised, with direct process-to-process communication enabled where possible[**?**]. A key aim developed during this design was to ensure that all critical IPC paths had a combined instruction footprint small enough to fit within the L1 instruction cache[**?**], improving cache locality and reducing execution latency when cache residency can be maintained. Furthermore, Remote Procedure Call (RPC) semantics[**?**] are supported, combining message passing and control transfer into a single operation. This reduces scheduling overhead by enabling direct handoff between client and server threads when a service is invoked.

The minimal kernel size of sel4 makes full-system formal verification realistically possible. To enforce the security properties specified in its proof, seL4 employs a capability-based access control model[**?**] as shown in Figure 2.3, which associates permission ownership with clients, rather than with objects. Capabilities are unforgeable tokens, initially provided by the kernel, where possession grants explicit rights to their corresponding kernel objects, instead of a server having a list of client permissions and clients having credentials to validate their requests with. This removal of ambient user authority eliminates 'confused deputy' type problems, in which a privileged component is mislead into abusing its authority on behalf of an unprivileged caller.

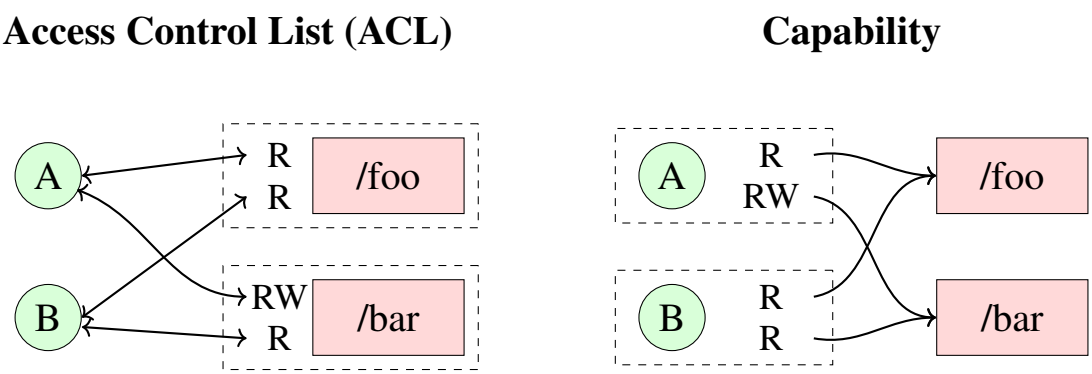


Figure 2.3: Comparison of ACLs and capability based access control models. Adapted from source: [2]

This distribution of authority also decentralises access control, removing the single-point-of-failure risk, and simplifies security management by eliminating identity permission checks. Following the principle of least privilege[**?**], capabilities are granted by the kernel only when required for operation and may only be transferred explicitly between processes *via* the kernel, ensuring that authority delegation is controlled and auditable.

2.2.3 Use Cases and Applications

seL4's principles of minimality and high assurance lend themselves particularly well to safety-critical and real-time systems, such as aerospace, automotive and medical devices, where correctness and isolation are crucial. However, it is also intended as a general-purpose microkernel[**?**] to enable high-performance systems, establishing that strong security guarantees are not a barrier to efficiency.

2.2.4 Microkit Framework for seL4

The seL4 Microkit is a lightweight operating system framework built on top of seL4, designed to simplify and accelerate system development, while still leveraging the microkernel's performance and security benefits. It achieves this by abstracting many complex, low-level seL4 mechanisms – such as capability management and system initialisation – and manages them automatically under a structured component model[**?**]. As such, systems are composed of Protection Domains (PDs)[**?**], each representing an isolated execution environment containing a thread, an address space and a capability space (CSpace).

This small number of simple abstractions lie very close to the implementation of seL4 itself, and as such, allow very efficient implementations. They also provide a clear and consistent structure for system design, which naturally encourages use of seL4 as it was intended.

To aid this abstraction simplicity, Microkit systems are defined statically through a system description[**?**], from which initialisation code is generated, as displayed in Figure 2.4.

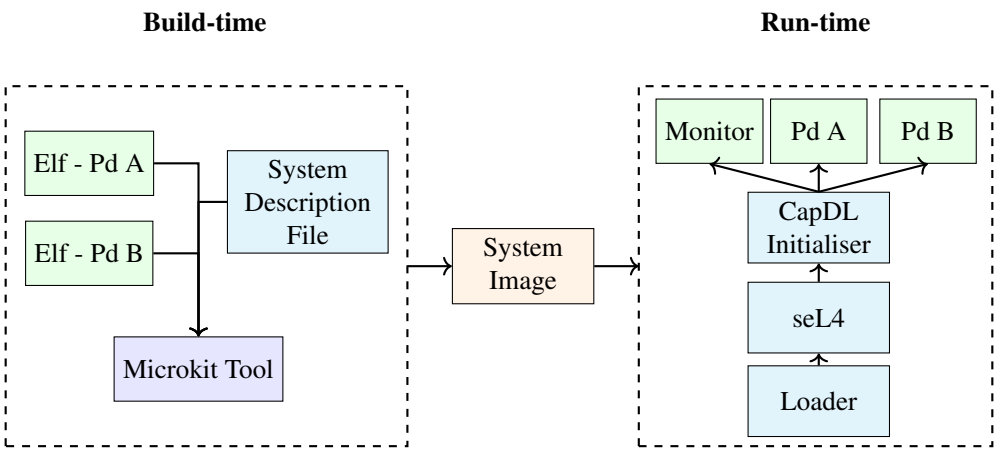


Figure 2.4: Build-time and run-time structures for a Microkit-based system. Adapted from source: [3]

This enables explicit specification of communication channels and shared memory regions between PDs, easing system construction and improving predictability. These communication channels between PDs are connected via well-defined entry points[**?**], as shown in Figure 2.5.

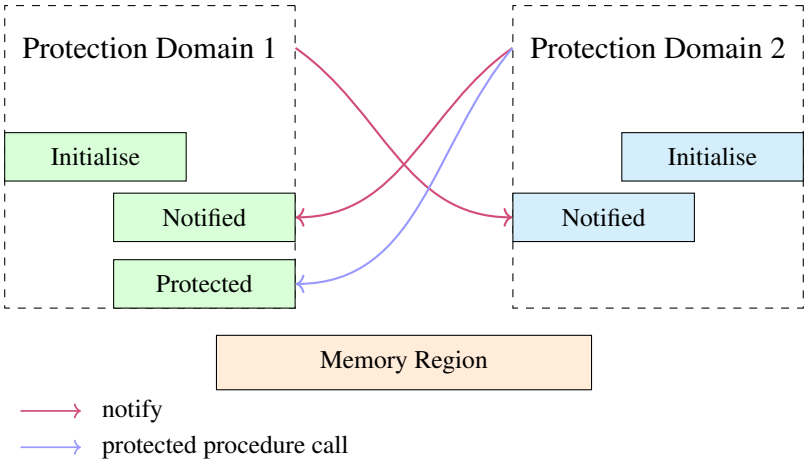


Figure 2.5: Communication channels between PDs. Adapted from source: [3]

Synchronous interactions are implemented using RPC communication, while asynchronous interactions are supported via notification objects[**?**], enabling event-driven signalling between components.

2.3 Client-Server Communication

2.3.1 Synchronous Model

Synchronous interfacing is a communication model in which the client blocks upon issuing a request and resumes execution only once a response is received from the server, as illustrated in Figure 2.6. This is most effective in systems where the server is highly available and response latency is low, such that blocking delays are minimised. It is therefore commonly employed in tightly-coupled environments, as they are built around assumptions of strong coordination and shared resources[**?**]. This enforced ordering of operations also significantly improves the predictability of interactions and reduces the system complexity[**?**], hence its use within the seL4 kernel.

It is also particularly appropriate for situations where any subsequent computation depends on the result of the request. However, when this dependency is absent, performance can be limited by this blocking mechanism due to client idling and high communication overhead for high-frequency operations, as only one may be completed per request.

2.3.2 Asynchronous Model

Alternatively, asynchronous interfacing decouples the client's request from server-side processing, allowing execution to continue independently from response handling, if further processing does not depend on the operation response, as shown in Figure 2.6. This enables significantly improved system throughput and resource utilisation potential. This effect is amplified in multiclient environments, where multiple clients may issue requests concurrently, avoiding the widespread blocking that arises when the server must service each request sequentially.

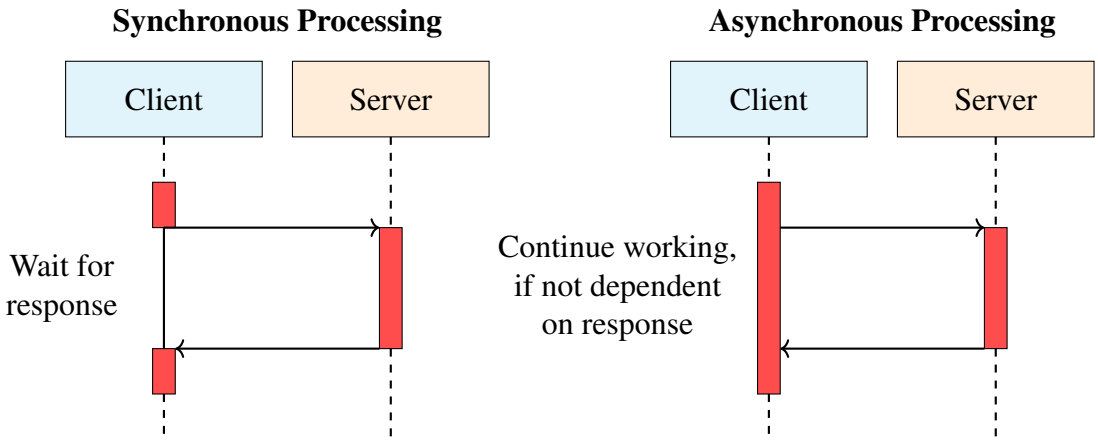


Figure 2.6: Comparison of synchronous and asynchronous communication models.

In addition, asynchronous designs enable batching, where multiple requests may be combined, sent and processed together. By reducing the number of IPC interactions, batching can significantly lower communication overhead and improve overall efficiency[**?**].

This model aligns naturally with microkernel design, where IPC is a dominant factor in performance[**?**]. As system components are structured as loosely-coupled, user-space services, minimising IPC-induced synchronisation delays between services is key to improving overall system efficiency.

However, these benefits come at the cost of increased design complexity. Supporting concurrent and non-deterministic usage patterns requires the use of queues and buffers to manage outstanding requests and responses. These structures must be carefully sized and managed (by both the client and the server) to prevent request loss or backpressure, preserving system elasticity. If insufficient buffering or queueing is available, the system may degrade towards synchronous behaviour, as clients are forced to wait for available capacity.

More generally, asynchronous communication shifts the overhead from client idle time to memory usage: latency due to blocking is reduced, but additional data structures and coordination are required to manage multiple requests.

2.4 File Servers in Operating Systems

2.4.1 Role and Functionality

A file server is a centralised operating system component responsible for managing persistent data storage and providing controlled access to that data for multiple clients, as described in Figure 2.7. Persistence is typically achieved through disk-backed storage, allowing data to outlive the execution of individual processes and, in many cases, system restarts. This enables clients to externalise state once it becomes relatively stable, reducing their memory usage while ensuring durability.

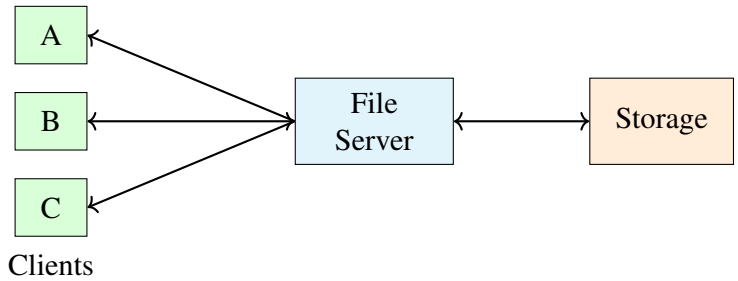


Figure 2.7: Multiclient file server structure.

By maintaining data within a shared, centralised repository, the file server facilitates implicit data sharing between clients without requiring explicit inter-process data transfer. This improves efficiency by reducing duplication and minimising communication overhead. However, such a shared access model requires robust mechanisms for access control and isolation. The file server must enforce policies that govern which clients can read, write, or modify specific files, ensuring security, integrity and consistency in a concurrent multiclient environment.

2.4.2 Abstractions

To facilitate efficient data storage and retrieval, a file system introduces multiple layers of abstraction over raw storage. The primary abstraction is the file, which represents a named, persistent sequence of bytes independent of its physical layout on the underlying medium. This logical abstraction enables interactions with the data as though it were stored contiguously, regardless of fragmentation or layout optimisations performed by the system.

In addition to the byte sequence itself, each file is associated with metadata which describes its properties. This metadata commonly includes attributes such as file size, access permissions and ownership[**?**]. Such information is essential for enforcing access control and supporting higher-level system functionality.

Client interactions with files are mediated through a well-defined set of parameterisable operations, typically including create, open, read, write, close and delete as a minimal interface[**?**]. These operations provide a consistent and reliable mechanism for managing the file lifecycle and accessing file contents.

To decouple user processes from file server internals[**?**], these operations reference files indirectly via a process-local 'handle', obtained through an open or create operation. These handles, commonly referred to as 'file descriptors', additionally encapsulate per-process state associated with an open file instance. This state includes attributes such as the access mode (e.g. read or write) and the current file byte offset, reducing the need to repeatedly specify this information across operations.

Furthermore, to support client management of multiple files, they are often organised hierarchically within *directory structures*, providing a namespace that maps human-readable names to underlying file objects. This organisation also allows easier metadata management and lookup operations for groups of files.

2.4.3 Storage and Organisation

File Layout

Unlike the logical view of a file as a contiguous byte sequence, physical storage does not generally store files contiguously. Doing so would make storage management inefficient, as operations such as file growth could require relocating the entire file's data if the next bytes are not free. As such, most file systems instead employ a block-based storage model[**?**].

In this model, storage is divided into fixed-size blocks, which can be allocated to files independently. A file is therefore represented as a collection of these blocks, with metadata structures maintaining the mapping between logical byte offsets and their corresponding physical locations. Block sizes (typically around 4KiB) are often chosen to align with virtual memory page sizes, allowing more efficient interaction between the file system and memory transfers[**?**].

While this approach improves flexibility and space utilisation, it introduces overhead. Sequential data may be physically non-contiguous, potentially reducing access performance due to increased seeking costs[**?**]. Furthermore, as all accesses occur at block granularity, the choice of block size represents a trade-off between space efficiency and operational overhead. Smaller blocks reduce wasted space when storing small files, but increase metadata overhead and the number of accesses required for large files.

Allocation Strategies

There are various methods through which these blocks are mapped to the contiguous file data.

Contiguous Allocation

The simplest strategy is contiguous allocation, where a file is stored in a sequence of adjacent blocks. This provides efficient sequential access and hence requires minimal metadata. An example implementation of this is shown in Figure 2.8.

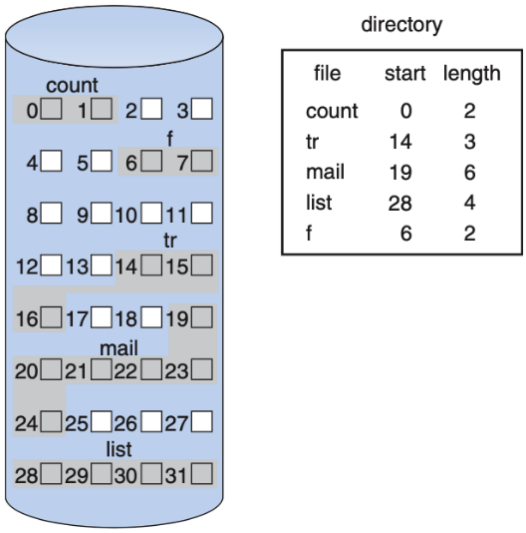


Figure 2.8: Example of contiguous block allocation. Source: [4].

Compared to a contiguous, non-block-based approach, it retains the same requirements for free space, and therefore suffers from similar limitations in file growth and susceptibility to external fragmentation. However, it improves upon this model by introducing fixed-size allocation units, allowing storage to be managed more flexibly at the block level.

Linked Allocation

Linked allocation avoids the limitations of contiguous allocation while maintaining relatively low metadata overhead per file. In this approach, each block contains a pointer to the next, as shown in Figure 2.9, allowing files to grow dynamically as any free block can be allocated.

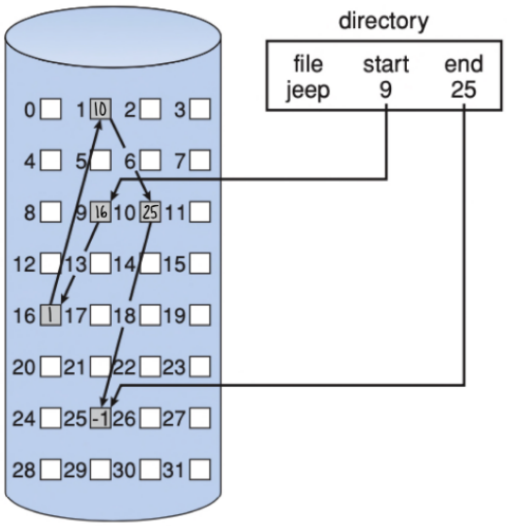


Figure 2.9: Example of linked block allocation. Source: [4].

However, this structure makes random access inefficient, as locating data at a given offset requires traversing the chain of blocks sequentially.

Indexed Allocation

Indexed allocation, which is the most commonly used approach[**?**], addresses the limitations of both contiguous and linked allocation. In this method, each file is associated with a metadata structure that stores references to its data blocks, allowing direct access to any portion of the file, as illustrated in Figure 2.10.

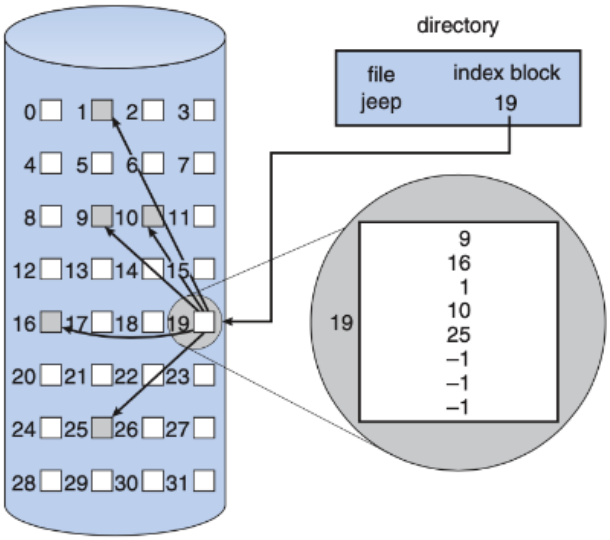


Figure 2.10: Example of indexed block allocation. Source: [4].

This enables both flexible file growth and efficient random access. However, this comes at the cost of increased metadata overhead, as additional storage and lookups are required to resolve block locations.

Metadata Organisation

Metadata may be organised either centrally or on a per-file basis. In centralised approaches, such as File Allocation Table (FAT) systems[**?**], a single global structure stores the mappings for all files. This is achieved using a linked allocation strategy with external pointers, where each entry in the FAT table corresponds to a block and contains a reference to the next block in the file, or an end-of-file marker. An example of this is shown in Figure 2.11.

This means per-file block metadata need only store the index of the file's starting block. While this approach is simple and centralises metadata management, it inherits the limitations of linked allocation.

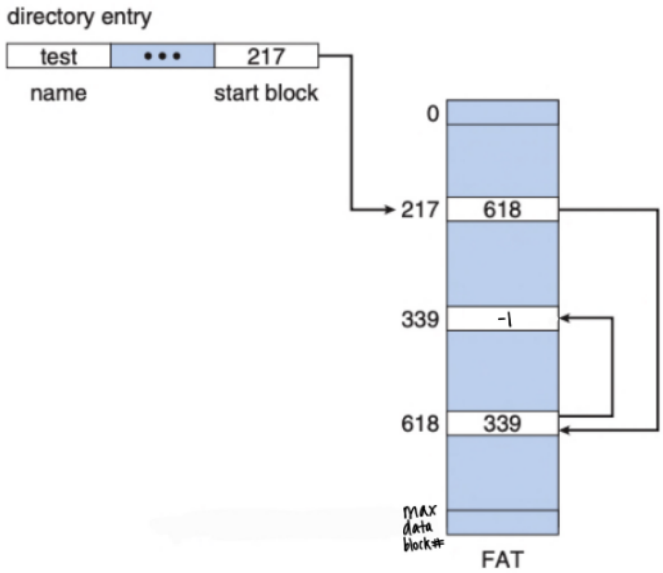


Figure 2.11: Example of FAT based file management. Source: [4].

Alternatively, per-file metadata strategies, such as inode-based policies used in POSIX-compliant systems[**?**], associate each file with a dedicated metadata structure that stores both file attributes and references to its data blocks. These references may be stored using a combination of direct and indirect pointers. Direct pointers reference data blocks directly, but as inodes are fixed-size structures, their number is limited. To support larger files, indirect pointers are used, where a pointer instead references a block containing further block addresses. This is illustrated in Figure 2.12.

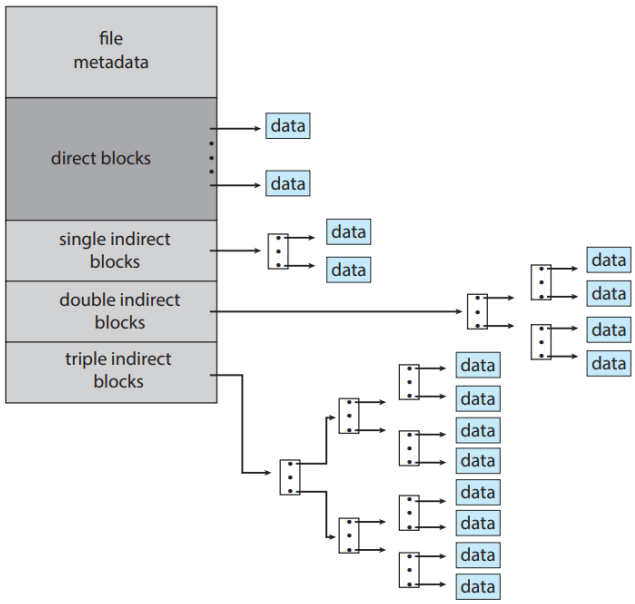


Figure 2.12: Example of inode based file management with indirect block indexing. Source: [4].

While this enables scalable file representation, it introduces additional levels of indirection, increasing the overhead required to resolve data locations.

Hybrid approaches of these are also common such as New Technology File System (NTFS)[**?**], which superseded FAT in Windows. NTFS employs per-file metadata structures similar to inodes, but represents file data using extents, storing ranges of contiguous blocks rather than listing each block individually, reducing metadata overhead for large, contiguous files.

Directory Structure

A directory is a structure that stores references to files rather than the file data itself, allowing files to be organised hierarchically through subdirectories, as illustrated in Figure 2.13. They are typically also implemented using block-based storage, where each directory consists of one or more blocks containing a list of directory entries.

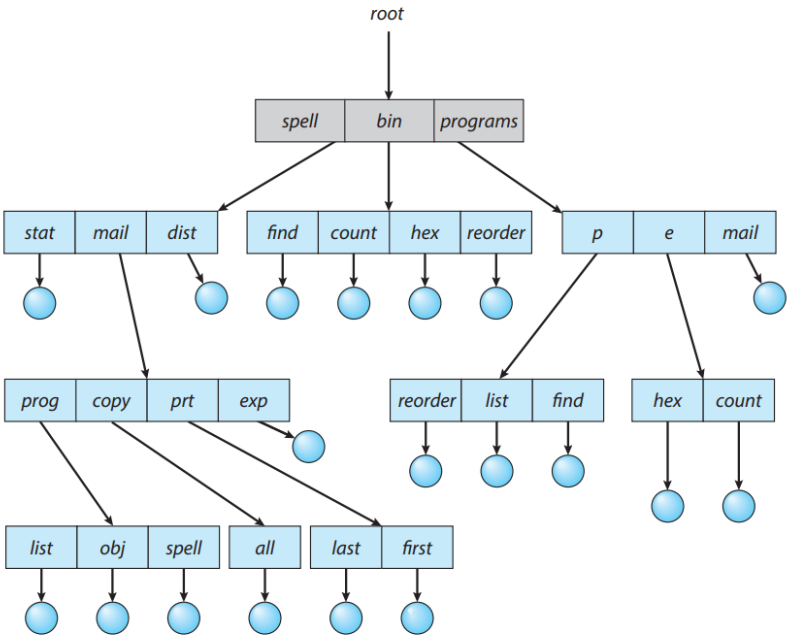


Figure 2.13: Example of a hierarchical directory structure. Source: [4].

Each directory entry is a structured record that represents either a nested file or subdirectory. These records are used to bind a human-readable name to the corresponding entry's metadata structure. This design is inherently recursive, since a directory entry can reference another directory whose blocks contain further lists of entries. Entries are then accessed via paths, where a path specifies a sequence of directory entries that must be traversed to resolve a name to its associated metadata.

2.4.4 Client Management

As previously discussed in Section 2.4.2, metadata stores information such as file ownership and access permissions. These are required to ensure that clients can control access to their files and prevent unauthorised modification or retrieval by others. At the simplest level, files may be designated as private, where only the owner has access, or public, where any client may access them.

More fine-grained systems define separate permissions for operations such as reading, writing and execution, allowing, for example, a file owned by one client to be private in terms of modification to other clients, while remaining public in terms of access. More advanced models, such as capability-based access control or access control lists (ACLs), as discussed in Section 2.2.2, allow permissions to be specified for individual clients or groups of clients, providing greater flexibility at the cost of increased complexity in storage and management.

Additionally, when multiple clients may have write access to a file, mechanisms may be employed to prevent race conditions – where concurrent updates may overwrite or invalidate one another – and ensure consistency, particularly for deletion operations.

2.5 Related Work

Existing file systems and I/O services are commonly implemented using synchronous request-response models, where operations block until completion[**?**]. While this simplifies reasoning and integration with existing software, it can limit concurrency and lead to inefficient resource utilisation. Recent developments, such as `io_uring` in Linux, have demonstrated significant performance improvements by enabling asynchronous I/O through submission and completion queues, reducing syscall overhead and allowing multiple outstanding operations[**?**]. However, this shift in execution model has introduced security concerns, as existing security mechanisms were not designed for such interfaces, leading to vulnerabilities arising from increased complexity and a larger attack surface[**?**].

In contrast, seL4 is a comparatively less scaled and more recent system, designed with strong security guarantees and a minimal trusted computing base. As a result, it lacks much of the legacy infrastructure that constrains the adoption of new interfaces in more established systems[**?**]. Furthermore, unlike some other microkernel designs, seL4 does not provide asynchronous primitives at the kernel level, instead relying on synchronous IPC. While this design performs well for individual interactions, it can limit concurrency in user-space services where operations may require multiple sequential IPC calls and have multiple clients.

Consequently, introducing asynchronous behaviour at the user level presents a promising approach to improving seL4 performance without compromising the kernel's design principles. However, there is limited existing work exploring asynchronous file server designs in seL4.

3 Design

This section describes the design of the file server, including both the interface through which clients interact with the system and the internal file system structure. As both components significantly influence how operations are issued and processed, their design plays a key role in overall system behaviour.

3.1 System Requirements and Constraints

The design of the file server is driven by both functional requirements and the constraints imposed by Microkit and seL4. These requirements define the intended behaviour of the system, while the constraints shape the design decisions necessary to achieve this within the given environment.

From a functional perspective, the system is required to provide a core set of file system operations, including creation, reading, writing and deletion of files, alongside support for directory structures and getting and setting of associated metadata such as permissions and file size. The system must present a consistent and simple interface to clients, ensuring that operations are executed correctly and that appropriate error codes are returned in response to invalid inputs or failure conditions.

A key requirement of the system is support for multiple clients operating concurrently. The file server must therefore be capable of handling requests from multiple independent clients, maintaining correctness and consistency in the presence of interleaved operations. In addition, the system must support both synchronous and asynchronous interaction models. This will enable direct comparison between traditional blocking request–response behaviour and the proposed asynchronous, batched approach.

Performance is a central non-functional requirement. In particular, the system is designed to minimise the overhead associated with IPC, which is a known bottleneck in microkernel-based systems[**?**]. The asynchronous design must therefore enable efficient batching of operations, reducing the number of required IPC interactions and improving overall throughput.

Several constraints significantly influence the system design. The Microkit framework enforces a statically defined system configuration to aid high-assurance and high-performance system development, meaning that all protection domains, communication channels and memory regions must be specified at build time. As a result, the number of clients and their associated resources are fixed, hindering the system's ability to adapt.

The IPC model provided by seL4 is inherently synchronous, with communication occurring through blocking interactions. This motivates the use of shared memory and notification mechanisms to enable non-blocking request submission and completion handling.

Hence, memory cannot be shared dynamically between protection domains, preventing the use of copy-free mechanisms that rely on on-demand sharing. Consequently, all data transfer must occur through these shared memory regions, requiring explicit buffer management within the interface design.

Further constraints arise from the execution environment. For development simplicity, the system operates with no access to persistent storage, resulting in a purely in-memory file system. Additionally, due to seL4 constraints, execution is limited to a single core, restricting the ability to exploit parallelism. This requires the system to rely on batching and efficient communication patterns, rather than parallel execution, to achieve performance improvements.

These requirements and constraints directly inform the design of the system, motivating the use of asynchronous batching, shared memory communication and a modular file system structure to balance performance, correctness and simplicity within the limitations of the Microkit environment.

3.2 IPC Design and File Server Interface

The file server interface defines how clients interact with the system to issue operations and receive results. As the interface determines how frequently and in what manner inter-process communication occurs, it plays a critical role in the overall performance of the system. Two models are considered: a synchronous interface, representing a traditional baseline for file server evaluation, and an asynchronous interface, designed to reduce communication overhead through batching and decoupling of request submission and processing.

3.2.1 Synchronous Model *(Baseline)*

This model represents a conventional microkernel interaction pattern, providing a simple and well-defined interface for issuing file system operations and receiving results.

The communication flow is as follows:

- A client issues a single operation via a synchronous IPC call and blocks until the request is completed.
- The server receives the request, processes the operation, and returns the result.
- The client then resumes execution.

This blocking behaviour enforces strict coupling between client and server execution. In this model, it is assumed any subsequent operation may depend on the result of the request, hence the client must do no further processing until the response is received. Consequently, client execution is effectively serialised with that of the file server, as shown in Figure 3.1, even if the client and server execute on different cores. Furthermore, the server itself only performs work when servicing requests and is otherwise idle.

As operations are strictly sequential, a single shared buffer per client is sufficient. Each request can reuse the same buffer, as its contents are consumed before the next operation is issued. This simplifies both memory management and interface design, as data is always read from and written to a consistent location.

To reduce overhead, non-data components of an operation may be passed directly within the IPC message. This avoids the need for complex parsing logic, as the shared buffer is reserved for file-related data such as filenames or read/write contents, while the IPC parameters encode the operation type and associated metadata, such as file descriptors or byte counts.

In a multiclient setting, blocking effects are amplified. If a client issues a request while the server is servicing another, it must wait not only for its own operation to complete but also for the server to become available, as shown in Figure 3.1. As a result, requests are effectively serialised across clients. Therefore, with more than 2 clients, the file server must maintain a queue of pending requests to ensure fair servicing, yet, this is bounded by the number of clients, limiting its complexity.

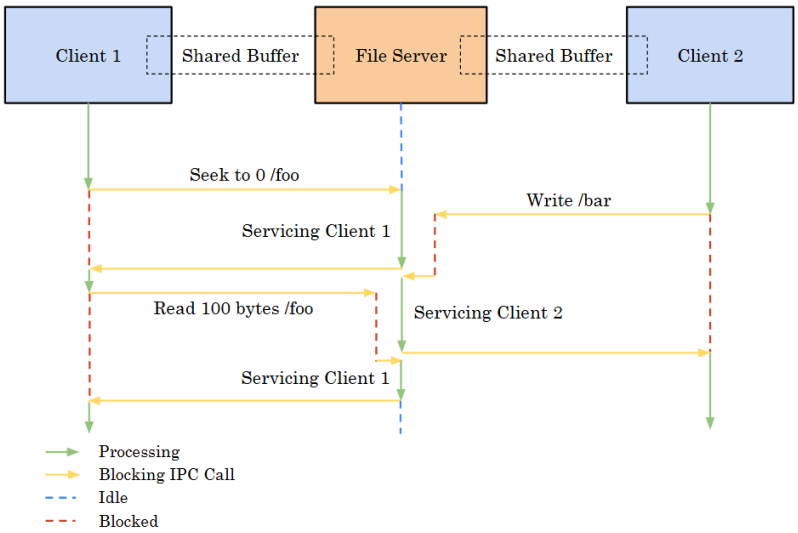


Figure 3.1: Synchronous multiclient-server interaction.

Consequently, this model permits no concurrency in relation to the file server, and clients spend a significant portion of time blocked, particularly as the number of clients increases. Additionally, IPC overhead is incurred on a per-operation basis, as each request requires a dedicated interaction with the server. This makes it unsuitable for workloads where large numbers of independent operations are performed, but provides a clear and well-defined baseline for comparison with the asynchronous design.

3.2.2 Asynchronous Model *(Proposed IPC Mitigation Strategy)*

The asynchronous model replaces the synchronous request–response interaction with a non-blocking, decoupled communication approach. While this increases the complexity of the interaction and the resources required, it enables significantly greater potential for performance improvement.

This non-blocking communication is achieved through a notification-based mechanism, rather than procedure calls. As a result, there is no requirement for a handshake or immediate response, allowing the client to issue a notification and continue execution without blocking, as illustrated by the continuous client execution in Figure 3.2. However, if the following computation relies upon the result of the requested operation, and no later processing may be moved forwards, the client will still have to wait. Yet, this model provides flexibility, enabling concurrency where possible while preserving correctness when dependencies exist.

When the server is idle, a notification triggers immediate processing of the corresponding request. Otherwise, unlike the synchronous system, where clients wait until the server becomes available to transfer their request, request submissions must be stored until they can be processed. As the server may be busy, this queuing cannot be managed internally by the server at the time of submission. Instead, each client is assigned a dedicated shared memory region with the server, used to queue outstanding requests. Notifications therefore carry no data, acting solely to signal that work is available, while the actual request data is retrieved by the server from these shared memory queues. As such, upon completing an operation the server polls these client queues to identify and service any pending requests that may have been submitted while it was busy.

To support efficient and correct operation, this shared memory must be carefully structured. As in the synchronous design, a separation between operation metadata and operation data is maintained. However, due to the absence of message-based data transfer, a queue of submission entries is required in addition to per-operation data buffers. To enable multiple outstanding requests without contention, a pool of submission buffers is used rather than a single shared buffer. This pool must be bounded to balance memory usage against achievable concurrency. Therefore, alongside the necessary metadata, each submission entry must include a reference to the operations associated buffer.

A corresponding completion queue and set of response buffers are provided for the server to return results. Separating submission and completion structures avoids the need for in-place modification of request data, which simplifies reasoning about ownership and correctness. This design ensures that each buffer has a clear single producer – single consumer relationship, reducing the risk of data corruption, and avoiding unnecessary allocation of buffers for operations that do not require them.

As operations are completed in-order per client, completion entries correspond directly to submitted operations, allowing the client to match responses to their original requests. To acquire these responses, a client may either poll the completion queue or wait for a notification from the server indicating that results are available. Each completion entry then contains the result of the operation, including any return value or error status, as well as an optional response buffer index, allowing the client to determine whether the request was successful.

By decoupling request submission from notification, this design implicitly enables batching. A client may enqueue multiple requests before notifying the server, allowing a single IPC event to trigger the processing of multiple operations. This reduces the number of IPC interactions required and amortises communication overhead across a batch, as illustrated by Client 1 in Figure 3.2. To maximise efficiency, the server processes all queued requests for a client before polling others. However, this must be bounded to prevent starvation, ensuring fair servicing across clients. To maintain correctness, queue entries are marked as ready explicitly by the client, rather than relying solely on queue occupancy, allowing the server to distinguish between valid and incomplete submissions.

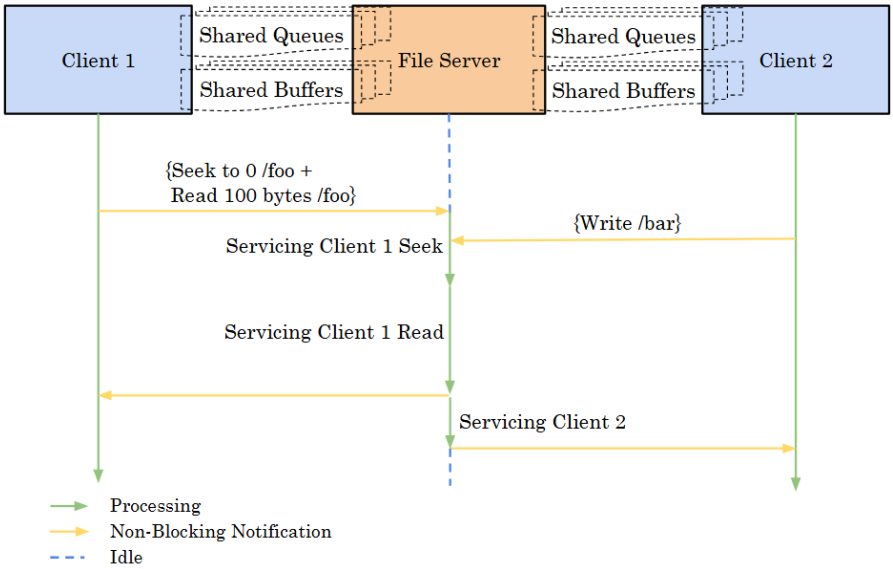


Figure 3.2: Asynchronous multiclient-server interaction.

While batching can increase the latency of individual operations, it reduces overall IPC frequency and context switching, significantly improving system throughput. Furthermore, the decoupled nature of the interaction allows client execution and server processing to overlap where possible, reducing unnecessary serialization and enabling more efficient utilisation of system resources.

The improvements can be understood through an analytical model of the system. Let a_n denote the time taken for a batch of n operations in the asynchronous system, then let x represent the internal processing time per operation and c_{a_n} represent the communication overhead for the batch.

Therefore, the total execution time for an asynchronous batch of operations can be expressed as:

$$a_n = c_{a_n} + nx \tag{3.1}$$

Hence, the average time per operation is:

$$\frac{a_n}{n} = \frac{c_{a_n}}{n} + x \tag{3.2}$$

This shows that as the batch size increases, the communication overhead is amortised across multiple operations, causing the per-operation communication cost $\frac{c_{a_n}}{n}$ to decrease. In the limit, the average operation time approaches the internal processing cost x, indicating that communication overhead becomes negligible.

This highlights the fundamental advantage of batching, as it allows the system to approach a performance bound determined primarily by computation rather than communication.

3.3 File Server

The following subsections discuss the internal design of the file system, including its data structures and organisation. The design focuses on providing a simple and efficient structure to support the required operations, while remaining compatible with both synchronous and asynchronous interface models.

3.3.1 Architectural Overview

The file server is structured as a set of modular components, each responsible for a distinct aspect of file system operation. Rather than treating the server as a single monolithic body of logic, functionality is decomposed into separate components for request handling, file system operations, runtime state management and storage management. Together, these provide a unified file system service to clients while maintaining clear internal separation of concerns, in keeping with core microkernel design principles. Furthermore, this modularity supports uniform error handling, allowing failures to be detected accurately, propagated naturally between components and returned to clients in a consistent manner.

At the highest level, the request handling layer forms the entry point into the server, receiving operations from clients and dispatching them to the appropriate file system logic. The file system core implements the semantics of individual file operations, coordinating access to the internal data structures required to service them. Runtime state management maintains transient execution state associated with active file handles, such as access modes and file offsets, while the storage management layer manages the underlying in-memory representation of file contents, metadata and allocation state. This decomposition is illustrated in Figure 3.3.

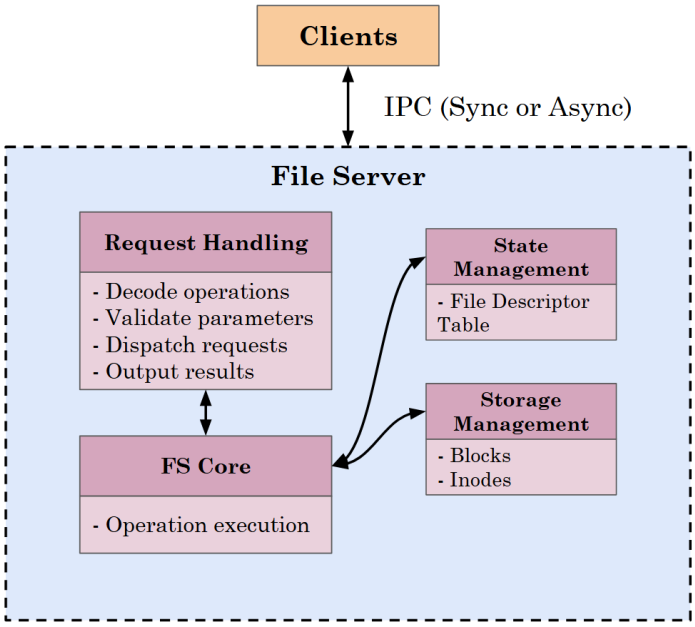


Figure 3.3: Architectural overview of the file server.

At a system level, the file server is responsible for servicing client file operations, maintaining correctness of file system state, managing storage resources, and coordinating access across multiple clients. Importantly, these responsibilities are independent of whether requests arrive through the synchronous or asynchronous interface, as both models share the same internal file system logic. This ensures that differences in measured performance arise from interface behaviour rather than differences in core file system operation.

This modular structure allows internal structures or algorithms to be modified without affecting the overall architecture, while supporting extensibility, as features such as persistent storage, more advanced allocation schemes or alternative caching strategies could be introduced without fundamentally redesigning the server.

Hierarchical file storage through a directory structure is supported, allowing files and directories to be organised using path-based naming. Directories may contain both files and subdirectories, enabling nested storage and traversal in a conventional tree-like structure. This was chosen over a flat namespace to support structured organisation and realistic file system behaviour.

The file server supports a set of operations intended to provide a sufficiently complete and practical file system interface for both functional use and evaluation. These include file and directory creation (Create File, Create Directory), data access (Read, Write, Seek), file lifecycle management (Open, Close, Delete), metadata and state queries (Get Size, Exists, Get Permissions, Set Permissions), and directory enumeration (List).

This set was chosen to provide a balanced interface spanning namespace management, data access, runtime file state and metadata handling. Together, these operations exercise the principal structures and responsibilities of the file server without introducing unnecessary complexity beyond the scope of this work.

3.3.2 Component Design and Interactions

Significant storage management is required to support file system operations, motivating separate structures for representing data, metadata, runtime state and namespace organisation, along with mechanisms to govern their interaction.

Blocks

Blocks form the fundamental unit of data storage in the file server. They represent fixed-size regions of memory used flexibly to hold dynamic file system data, including file contents, directory entries and, where required, block indices for indirect blocks. Using a unified block abstraction allows storage to be allocated according to demand, rather than partitioning memory into separate fixed regions for different data types. This provides greater flexibility, allowing files and directories to grow as required while also avoiding unnecessary reservation of space when demand is low.

A fixed block size was chosen to simplify allocation, indexing and data extension. A size of 4 KiB was selected as a practical balance between storage granularity and management overhead. While fixed-size allocation means all operations occur at block granularity, potentially wasting space when storing very small files, this was accepted in exchange for more efficient allocation management and simpler indexing. As the system is modular, however, this block size remains configurable.

Blocks are managed through an block allocation table, as illustrated in Figure 3.4. This maintains one entry per block, with each entry storing a flag indicating whether the corresponding block is free or allocated. This enables rapid assignment and release of blocks, simplifying rearrangement of file system data.

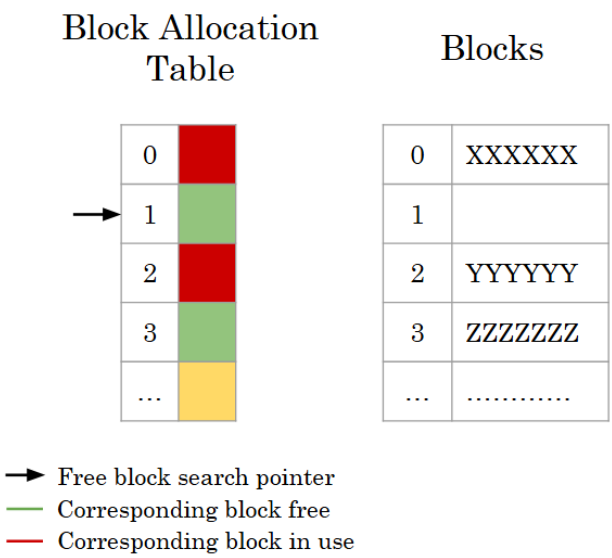


Figure 3.4: Block allocation table structure.

All blocks are stored contiguously and are accessed by index into the block pool, with these indices being stored within inodes.

inodes

Inodes provide the central metadata structure of the file system, representing either files or directories and linking their logical identity to the underlying storage by holding both entry metadata and block references. This structure was chosen to separate metadata from data storage, simplifying management of file system state. It also provides a uniform representation for file system objects, reducing special-case handling. Furthermore, resolving persistent objects independently of their names avoids repeated path-based lookup.

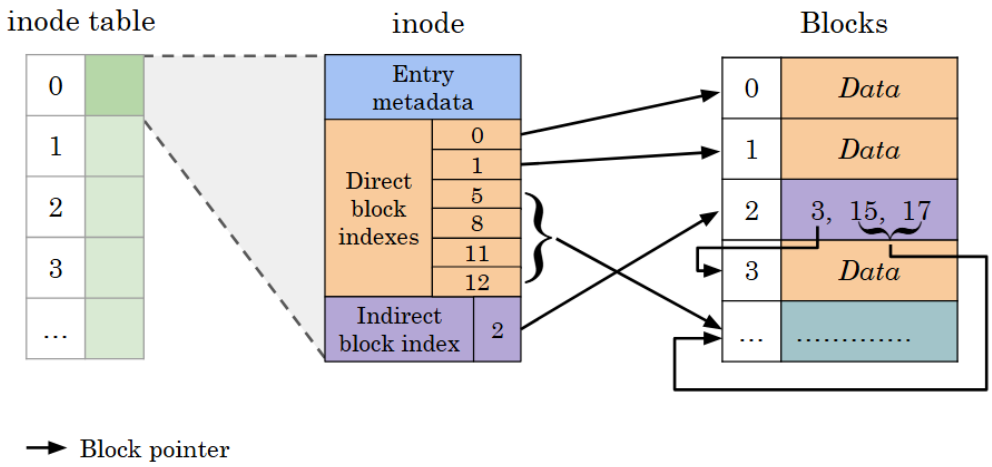


Figure 3.5: inode and block reference structure.

This metadata must hold a variety of information to allow rich and secure system functionality. To avoid requiring a separate allocation table, each inode includes an in-use flag to enable rapid allocation and deallocation. The inode type (file or directory) is also stored, ensuring associated data is interpreted appropriately and supporting valid file traversal. Ownership and access permissions are additionally stored to support multiple clients safely.

Inodes store block references either directly or, where additional references are required, through indirect blocks, as shown in Figure 3.5. This allows files to grow beyond the capacity of direct block references while preserving a uniform access model, although very large files may incur modest additional access overhead due to indirect reference resolution.

To aid efficient data access and ensure only initialised storage is accessed, an inode also stores the file size, indicating the extent of valid file data. Together with the block references, this supports Read operations through resolution of valid file data and Write operations through access to existing blocks, while determining when file extension requires allocation of additional blocks.

This design introduces an additional level of indirection between file system objects and their underlying data, but this was accepted in exchange for the flexibility, uniformity and scalability the inode structure provides.

Directories

Directories provide the namespace structure of the file system, enabling hierarchical organisation through mappings from names to inode references. They are represented using the same inode and block structures used elsewhere in the file system, reusing existing abstractions rather than requiring a separate storage mechanism, thereby preserving flexibility in data use and storage efficiency.

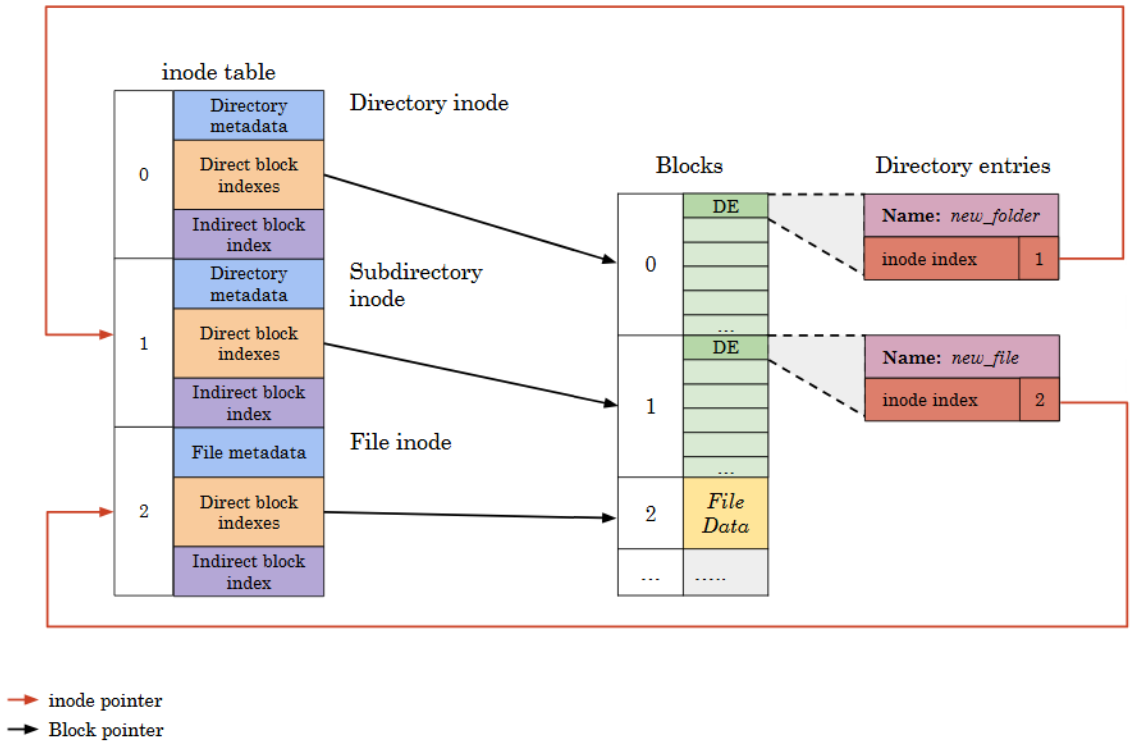


Figure 3.6: Directory structure via inodes and blocks.

A directory is represented by a directory inode whose associated blocks store a list of nested files and subdirectories as directory entries. Each directory entry is a structure linking an entry name to its respective inode via an index, as shown in Figure 3.6.

This recursive structure enables hierarchical namespace organisation through arbitrary nesting of directories and files, supporting structured management beyond what would be possible in a flat namespace. It also provides inherent path-based access, where successive path segments are resolved by searching the entries of the current directory, descending into a child directory when a matching entry is found, until the target inode is reached, or a path is deemed invalid. This traversal mechanism is used by file system operations to inspect and modify directory entries, enabling Create to enforce valid paths and prevent duplicate names within a directory, while supporting recursive directory deletion through traversal of all contained entries to ensure no unreachable data is left allocated.

For general use, a root directory is created during system initialisation, through which all files and directories are organised upon, hence, may not be deleted. Special directory entries such as . and .., as well as symbolic links, are not supported, reducing namespace complexity and avoiding additional traversal cases.

This structure was chosen to support path-based access and structured file lookup, allowing files to be located through traversal rather than requiring a flat naming model. However, as path resolution requires directory traversal, lookup incurs some overhead, particularly due to linear directory scans and possibly deeply nested paths. Linear scanning was chosen over more complex indexed lookup structures to reduce management overhead and implementation complexity, which was considered appropriate for the expected scale of the file system, while not affecting comparisons of the synchronous and asynchronous communication models.

File Descriptors

File descriptors maintain runtime state for open files, providing a layer of indirection between client operations and persistent file metadata. As shown in Figure 3.7, a file descriptor links a client-visible handle to the inode representing the underlying file, while additionally storing the transient state required for continued access.

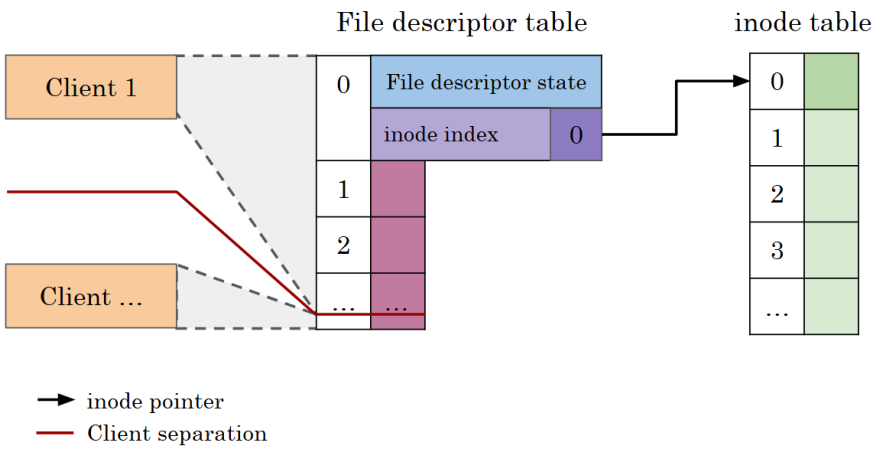


Figure 3.7: File descriptor structure.

This state includes a reference to the associated inode, the current file offset, and the mode in which the file was opened. Separating this runtime state from inode metadata allows information specific to an active open instance, rather than the file itself, to be maintained, supporting multiclient access. This is additionally useful for operations such as sequential reads or writes, where file position may persist across operations and need not be re-specified.

File descriptors are created and returned to a client when a file is opened, and are subsequently used by operations such as Read, Write, Seek and Close to reference the open file. This improves efficiency by avoiding path resolution for every operation, as the file descriptor is used to link directly to the associated inode.

This design does introduce some overhead in maintaining descriptor state, but this is modest relative to the performance benefits it provides. Fixed-size file descriptor regions do, however, impose a bounded number of simultaneously open files per client.

3.3.3 Request Processing Flow

File system requests follow a common processing path regardless of operation type. Once a request is received by the file server, it is first decoded to determine the requested operation and extract any associated parameters. Basic validation is then performed, such as verifying the operation code is recognised, the required parameters are present and checking that any referenced file handles or paths are correctly formed.

Following validation, the request is dispatched to the appropriate file system logic. Although the specific structures accessed differ by operation, all requests follow the same general pattern of interacting with the file system core, consulting or updating the relevant runtime state, accessing the underlying storage structures as required, and generating a result to return to the client.

The general flow is:

- Receive request
- Decode operation
- Validate parameters
- Service operation
- Access internal structures
- Update state if required
- Generate response

For example, Read request follows this general structure by validating the supplied file handle exists and is a file, consulting the open file state to determine the current offset, resolving the relevant storage blocks containing the requested data, and returning the result while updating the file offset. Operations such as Create and Write access and modify different state, but follow the same overall dispatch and servicing model.

This shared processing path was chosen to reduce duplicated logic, ensure consistent validation across operations, and simplify reasoning about correctness within the file server.

3.3.4 Multiclient Support

Multiclient support is provided through a combination of per-client runtime state, client isolation and centralised coordination through the file server. This allows multiple clients to access the file system concurrently while preserving correct behaviour and separation of client context.

Per-client state is maintained through both persistent and runtime structures. Ownership and access permissions are stored within inode metadata, allowing access control decisions to be enforced consistently regardless of which client issues a request. Permissions are represented using read, write and execute flags, allowing the file server to determine whether a client may perform a requested operation. Ownership associates each file or directory with the client that created it and restricts operations such as permission modification. Ownership also supports private access: if a permission flag is unset, only the owner may perform the corresponding operation, whereas if the flag is set, any client may do so.

Runtime state associated with open files, such as file offsets and access modes, is maintained in file descriptors. To preserve client independence, each client is assigned a separate region of the file descriptor table, with a local maximum number of open files. This ensures that one client's open file state cannot interfere with that of another. This separation of runtime state provides client isolation even when multiple clients access shared resources. While multiple clients may access the same underlying file, each does so through its own client context, maintaining independent offsets, access modes and descriptor state.

Consistency of shared resources is preserved through centralised mediation by the file server; all file operations pass through the server and are serviced atomically, with updates to shared file system structures serialised.

A particular case of shared resource consistency arises when one client deletes a file that remains open by others. In this case, deletion is deferred until the final client closes the file, ensuring that active references remain valid and preventing removal of data still in use.

4 Implementation

This section describes the implementation of the file server, building upon the designs presented in the previous section. It details how both the interface and file system components were realised in practice, including data structures, algorithms, communication mechanisms and API abstractions.

4.1 Development Environment and Tools

The system was implemented in C using the Microkit framework for seL4, specifically version 2.0.1 of the Microkit SDK[**?**]. The build process was managed using a Make-based system, which coordinated compilation, linking and generation of Microkit system images.

All development and testing were conducted in an emulated environment using QEMU[**?**], targeting the `virt` generic virtual machine model. This allowed rapid iteration without requiring physical hardware, though it limited evaluation of hardware-specific performance characteristics.

Debugging and verification were primarily performed using logging via serial output, enabling inspection of system behaviour during execution. Additional support tools included Git[**?**] for version control and reproducibility of development progress.

4.2 Microkit Framework Setup

The system is built using the Microkit framework for seL4, which provides a declarative mechanism for defining execution structure, memory layout and inter-process communication[**?**]. This is specified using an XML-based description, which is used to generate a complete system image comprising multiple protection domains. A single client example of this system definition is shown in Listing 4.1.

Each protection domain represents an independent user-space process with its own address space and execution context. Hence, separate protection domains were defined for the file server and each client. Program images were statically assigned to each domain, and memory regions were explicitly declared and mapped into their address spaces at specified addresses. A dedicated memory region for file system state was declared and mapped solely to the server. Additionally, for each client, regions to hold submission and completion queues and associated buffers were declared and mapped into both the file server and the corresponding client, each with the appropriate permissions, allowing shared access.

Alongside this shared memory, IPC between protection domains was established using channels, as detailed in Section 2.2.4, with each client being connected to the file server. By defining memory regions and notification channels through Microkit, capability management of all objects is handled automatically by the framework, significantly reducing the complexity of managing and securing IPC.

```
     <!-- File Server Memory Region -->
    <memory_region name="fs_memory" size="0x10133000" />

    <!-- File Server Protection Domain -->
    <protection_domain name="file_server" priority="1" >
        <program_image path="file_server.elf"/>

        <map mr="fs_memory" vaddr="0x30000000" perms="rw" cached="true"
          setvar_vaddr="fs_memory_base"/>

        <map mr="client_0" vaddr="0x40133000" perms="rw" cached="true"
          setvar_vaddr="clients_memory_base"/>

    </protection_domain>

     <!-- Client Memory Regions -->
    <memory_region name="client_0" size="0x81000" />

    <!-- Client Protection Domains -->
    <protection_domain name="client_0" priority="0" >
        <program_image path="client0.elf"/>
        <map mr="client_0" vaddr="0x30000000" perms="rw" cached="true"
          setvar_vaddr="fs_data_base"/>
    </protection_domain>

    <!-- Client Communication Channels -->
    <channel>
        <end pd="file_server" id="0"/>
        <end pd="client_0" id="0" pp="true"/>
    </channel>
```

Listing 4.1: Microkit system definition.

As Microkit systems are statically defined, the number of clients and their associated resources must be specified at build time. To support varying workloads, a script was developed to generate the system description for a configurable number of clients automatically, ensuring consistency while avoiding manual duplication.

In addition to structural configuration, Microkit also requires specification of priority and scheduling parameters for each protection domain. These priorities determine the order in which domains are scheduled by the seL4 kernel, directly influencing system responsiveness and fairness. The file server was, therefore, assigned a higher priority than the client domains to ensure that incoming requests were serviced promptly, reducing queuing latency and preventing clients from stalling due to delayed processing. Client domains were assigned equal priorities relative to one another to maintain fairness across request sources. This scheduling configuration was particularly important in the asynchronous design, where the server must efficiently respond to notifications and process queued operations without being preempted excessively.

4.3 Interface

4.3.1 Synchronous Server Model

This is the simpler of the two models, and as such, was implemented first to validate the correctness of the file system, before introducing the additional complexity of the asynchronous design.

Client-Server Communication

Microkit's protected procedure call (`microkit_ppcall`) provides both the blocking semantics and message passing we require within a single operation. It transfers control from the calling thread to the callee, suspending the caller until the operation returns. As such, the callee must execute at a higher priority than that of the caller, which aligns with our design as we already have the file server at the highest priority for scheduling preference.

In addition to control transfer, `microkit_ppcall` supports the transmission of up to 64 words of data in both the request and response. As file names and data are handled via shared buffers, this capacity is sufficient to encode the operation code and associated parameters or return values. Message size is configurable, allowing each operation to transmit only the required data. Consequently, the server must validate the message size after decoding the operation to ensure all necessary parameters are present.

The client request flow proceeds as follows:

- The client writes any required data (such as file names or file contents) into its shared buffer.
- A message is constructed containing the operation code and relevant parameters.
- The client invokes the file server via `microkit_ppcall` on its dedicated channel, passing the constructed message.
- The client thread is automatically suspended until the operation completes.

When control is passed to the server, execution begins at a designated entry point. This is defined as the function signature:

```
microkit_msginfo protected(microkit_channel channel,
                           microkit_msginfo msginfo
                          )
```

Since all clients share this entry point, the `channel` parameter is used to identify the calling client. These channel identifiers are statically assigned in the Microkit system configuration and cannot be modified at runtime, ensuring that requests are correctly attributed.

Alongside extracting the operation code and parameters from the message the server may also access the client's shared buffer. All client buffers are mapped into the server's address space at specified virtual addresses during system initialisation. By arranging the client buffers sequentially - in the same order as the channels, and hence the client IDs - the server can treat them as an array and index into it directly using the client's channel identifier to locate the appropriate buffer.

Once the operation is complete, the server constructs a response message containing the result code and any return values, and returns it from the protected function. This automatically delivers the response to the corresponding client and resumes the client thread, which can then inspect the result and any updated buffer contents.

API

To simplify usage and ensure consistent interaction with the file server, the communication and buffer management logic is encapsulated within a set of wrapper functions. These abstractions hide the details of message construction and extraction, buffer usage, and IPC invocation.

To reduce the number of parameters required for these library calls, each operation has a specific function that can be called, which accepts only the parameters required for that operation and returns an operation unique struct for maximum access simplicity.

For example, the `send_open_file_request` function takes the desired access mode, a pointer to the file name, and the address of the client's shared buffer. It copies the file name into the buffer, constructs a new message containing the `OPEN` operation code and mode parameter, and invokes the file server. Upon completion, it forms a `result_code_file_id` struct with the result code from the operation and the file ID returned. This struct is then returned to the client, allowing the outcome to be easily interpreted.

4.3.2 Asynchronous Server Model

This is the more complex of the two models and was developed after the synchronous implementation to reduce IPC overhead through asynchronous, queue-based communication.

Client-Server Communication

Rather than transferring control directly to the file server for each operation, the asynchronous model implements communication through shared memory and notification-based signalling. Clients submit requests by writing to shared data structures and continue execution until they explicitly check for completion.

Each client is assigned a submission queue, a completion queue, and corresponding buffer pools for transferring file-related data. Submission buffers are used to store request data such as file paths or write contents, while completion buffers hold data returned by the server. Allocation tables are maintained for both buffer pools to track usage. A full operation flow example is displayed in Figure 4.1.

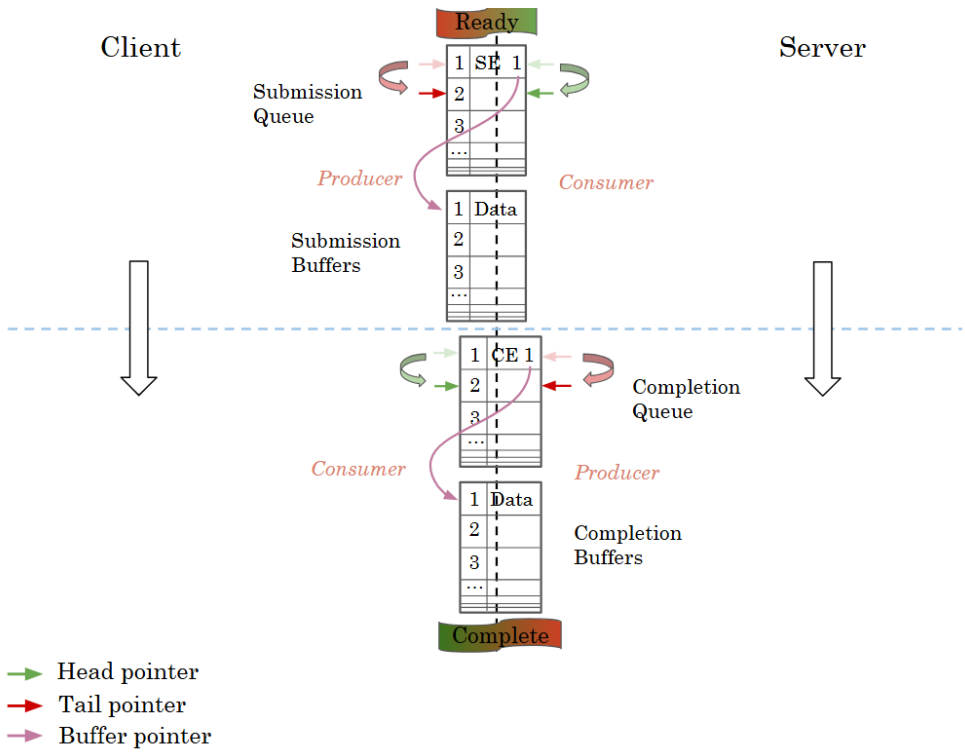


Figure 4.1: Asynchronous client-server communication via shared queues and buffers.

For operations requiring data transfer, the client first acquires a submission buffer by locating a free entry in the allocation table, as shown in Figure 4.2. If no buffer is available, the client must wait until its previously issued operations complete and their resources are released.

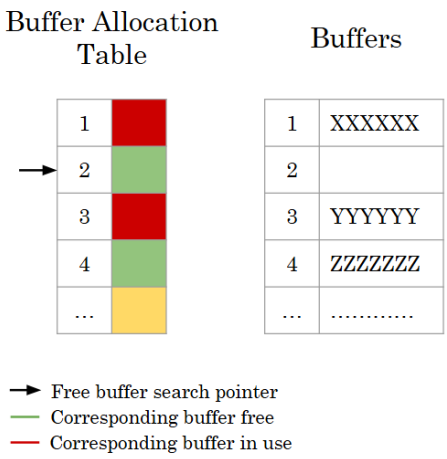


Figure 4.2: Buffer management in asynchronous communication.

Once a buffer is acquired, the client writes the required data and constructs a submission queue entry describing the requested operation, including its parameters and the associated buffer index. The structure of this entry is shown in Listing 4.2.

```
struct submission_queue_entry
{
    operation_t operation_code;
    uint32_t parameter1;
    uint32_t parameter2;
    size_t buffer_index;
} typedef submission_queue_entry_t;
```

Listing 4.2: Submission queue entry structure.

Submission and completion queues are implemented as fixed-size circular buffers, each maintaining head and tail pointers. Using fixed-size entries allows direct indexing via the head and tail pointers, simplifying access and improving efficiency. Yet, this requires all entries to share a common structure, meaning some fields may be unused for certain operations. As the size of each entry is small, however, this overhead has minimal impact on overall memory usage.

New entries are written at the tail, while entries are consumed from the head. The producer updates the tail pointer after adding entries, and the consumer updates the head pointer after removing them. Before adding or removing entries, the state of the queue must be checked to ensure it is not full or empty, respectively, by comparing the head and tail pointers. As a result, these pointers must be shared between the client and the server.

As Microkit only allows memory sharing at page granularity, with a minimum mapping size of 1 KiB, each client is allocated a contiguous region of shared memory with the file server, simplifying the static system description. This region is sized to accommodate all required data structures, including queues, buffers, allocation tables, pointers, and control flags.

This memory is organised as a single structured layout, as defined in the file server API. This also allows the relative sizes of internal structures to be adjusted without affecting the overall interface. The structure is shown in Listing 4.3.

Similarly to the synchronous setup, all client memory regions are mapped into the file server's address space at specified virtual addresses during system initialisation. As these regions are of uniform size and arranged sequentially, the server can index directly using the client's channel identifier to access the corresponding client state. Encapsulating all client state within a single structure simplifies function interfaces, as only a pointer to this structure needs to be passed, enabling a modular implementation of the file system.

```
struct client {
    size_t submission_queue_head;
    size_t submission_queue_tail;
    size_t completion_queue_head;
    size_t completion_queue_tail;
    client_flags_t flags;
    submission_queue_entry_t submission_queue[MAX_QUEUE_ENTRIES];
    completion_queue_entry_t completion_queue[MAX_QUEUE_ENTRIES];
    bool submission_buffer_table[NUMBER_OF_BUFFERS_PER_CLIENT];
    bool completion_buffer_table[NUMBER_OF_BUFFERS_PER_CLIENT];
    buffer_t submission_buffers[NUMBER_OF_BUFFERS_PER_CLIENT];
    buffer_t completion_buffers[NUMBER_OF_BUFFERS_PER_CLIENT];
    uint8_t padding[CLIENT_DATA_PADDING_AMOUNT];
} typedef client_t;
```

Listing 4.3: Client shared data structure.

Once the desired number of submission entries have been added, the client sets its ready flag and signals the server using Microkit's `microkit_notify` function. This sends a notification along a statically defined channel without blocking the client, allowing it to continue issuing further requests or performing other work. This ready flag allows the server to efficiently poll clients when required and ensures that only clients that have explicitly indicated pending work are serviced.

If the server is idle, receiving a notification causes its thread to be scheduled. Execution begins at the designated entry point:

```
void notified(microkit_channel ch)
```

The `channel` identifier is used to determine which client issued the notification, allowing the server to access the corresponding shared memory region. The server then checks that the client's submission queue is valid and begins processing any pending entries.

To service an operation, the server removes the next entry from the submission queue, incrementing the head, and extracts its parameters. If the operation requires a completion buffer, one is acquired using the same allocation mechanism as the client. If no buffer is available, the operation fails and an appropriate return code is generated. Otherwise, the operation is performed, and any result data is written to the allocated completion buffer.

A completion queue entry is then constructed containing the result code, any return values, and the buffer index. This entry is added to the completion queue. The structure of this entry is shown in Listing 4.4.

```
struct completion_queue_entry
{
    uint8_t return_code;
    uint32_t result1;
    uint32_t result2;
    size_t buffer_index;
} typedef completion_queue_entry_t;
```

Listing 4.4: Completion entry structure.

Once the operation is complete, the submission buffer is released by marking its entry in the allocation table as free, as its contents have been consumed. The server then continues processing entries from the submission queue until it is empty or a predefined per-client batch limit is reached, preventing any single client from monopolising the server.

When processing is complete, the server updates the client's flags to indicate that results are available, and that no outstanding requests remain (if processing was not terminated early), before sending a notification to wake the client if it is idle.

As notifications are not queued, the server cannot rely on them as the sole trigger for processing requests. After servicing a client, it polls all clients by checking their ready flags. This ensures that requests are not missed if a notification arrives while the server is already processing work. The server iterates over all clients, servicing any with pending requests.

On the client side, once notified or when explicitly checking for completion, the client removes entries from the completion queue and processes the results. After consuming any returned data, it releases the corresponding completion buffers by updating the allocation table, making them available for future operations.

API

Much of this communication complexity is abstracted away by library functions, providing a simple and consistent interface for interacting with the file server. As in the synchronous implementation, each file system operation is exposed through a dedicated wrapper function.

For example, `send_open_file_request` takes a file name, the desired opening mode and a pointer to the client's shared state structure. It first attempts to acquire a free submission buffer, failing if none are available, and then copies the file name into that buffer. Using the acquired buffer index and the supplied parameters, it constructs a `submission_queue_entry_t`, inserts it into the submission queue if space is available, and advances the submission queue tail. An appropriate result code is then returned to the client to indicate whether submission was successful.

As request submission is non-blocking, additional helper functions are required both for notifying the server and for collecting results. Two notification mechanisms are provided. The first uses `microkit_notify` to signal the file server and then returns immediately, allowing the client to continue execution when further independent work is available. The second is intended for cases where subsequent processing depends on completion of the submitted operation or operations. In this case, `microkit_ppcall` is used to suspend the client thread until the server has processed the request, similarly to the synchronous system, but without transferring request data. This is more efficient than busy waiting when no further useful work can be performed. Both the `notified` and `protected` server entry points then delegate to the same `service_client` function, ensuring that request handling remains identical regardless of how servicing was triggered.

If the client does not block when notifying the server, it must later check whether results are available. This can first be done by inspecting the `complete` flag. However, if multiple operations were submitted as a batch, the flag may be set before all expected results have been completed, in which case the completion queue head and tail pointers must also be compared. As no useful work may remain while waiting for these results, dedicated wrapper functions were implemented to either busy wait on these conditions or yield execution, returning only once the requested operations have completed.

Results are then retrieved using `get_next_completion_entry`. After any returned data has been consumed, the associated completion buffer may be released by the client through a dedicated wrapper function.

4.4 File Server

This section details the realisation of the file server design, covering implementation of its internal structures and the mechanisms used to handle file system operations.

4.4.1 Data Structures and Access Mechanisms

The file server is allocated a large chunk of memory for internal use during initialisation through the Microkit setup, which is then partitioned using statically defined constants into the internal data structures described in this section. This partitioning primarily determines the relative space assigned to inodes, blocks and the BAT, and therefore defines key system scale properties.

For example, increasing inode capacity allows more files and directories to be represented, but reduces available block space and therefore the potential size of stored file data. These allocations must therefore be balanced carefully. As this partitioning is performed within the file server itself, however, these values may be modified easily, providing flexibility in configuring the system.

Blocks

Blocks are implemented as fixed-size regions within a contiguous memory pool, with allocation managed through the BAT. This is realised by structuring the block-data region of the memory pool as an array of elements of size `BLOCK_SIZE`. The number of blocks that fit within this region is recorded, and a Boolean array of equal size is created to form the BAT. Each block is therefore associated with a corresponding index, which is used both to access block data directly from the memory pool and to update the corresponding BAT entry during deallocation, both in constant time.

As all blocks share the same representation, any block may store file data, directory entries or indirect block indices. Allocation for all such uses is performed through a linear scan of the BAT for an unset entry indicating a free block, while deallocation simply marks the corresponding BAT entry as free.

inodes

Inodes are implemented as fixed-size metadata structures, each representing a file system object and linking it to its associated storage blocks. To enable direct access, these are stored contiguously in an inode table. The inode structure is shown in Listing 4.5.

```
struct i_node
{
    size_t entry_size;
    size_t blocks_used;
    size_t block_indices[DIRECT_BLOCKS_PER_INODE + 1];
    uint8_t mode;
    uint8_t owner_id;
    uint8_t padding[2];
} typedef i_node_t;
```

Listing 4.5: Inode structure.

The `entry_size` field stores the exact number of bytes of file data, or the number of entries within a directory, returned when queried through the Get Size operation.In contrast, `blocks_used` limits reads and indicates when additional blocks are required for writes. This must be stored separately from `entry_size`, as directory entries may be deleted, meaning a directory's allocated blocks need not be full, so the number of blocks that must be scanned to reach all child entries cannot be inferred from `entry_size` alone. This additionally allows excess blocks to be identified and freed through defragmentation.

Blocks are referenced both directly and through single indirection. Space for eleven direct block references was chosen to minimise inode size while allowing reasonably large files and directories before indirect access is required, providing 44 KiB of directly addressable storage. For simplicity, and given the intended scale of the file system, only one indirect block is supported. This still allows substantial file growth, as a single indirect block may hold up to 128 additional block references. These capacities are defined through statically configured constants, allowing simple modification of system limits if requirements change.

To maximise block availability, block index fields are initially empty other than the first direct block, as every entry is allocated an initial block on creation to ensure immediate storage availability. Additional block indices are populated only as growth requires, allowing later access to those blocks for both use and deallocation during deletion. As long as the number of allocated blocks does not exceed the maximum number of direct pointers, newly allocated block addresses can be stored directly in the inode. Once this limit is exceeded, an indirect block must first be allocated and its address stored in the inode's indirect block field. Any further block addresses are then stored in the indirect block rather than in the inode itself.

To reduce space overhead, the deletion required flag, in-use flag, inode type and permissions are packed into a single byte, which is set and read using bit masks, as shown in Listing 4.6. Padding is also included to support efficient alignment of inode entries in memory.

```
    i_node->mode = IS_DELETED_BIT << 5 | PERMISSION_BITS << 2 | DIRECTORY_BIT
        << 1| IN_USE_BIT;
```

Listing 4.6: Inode mode field bit packing.

Directories

Directories are implemented through directory inodes whose associated blocks store directory entries, realising the hierarchical organisation described in the design. The directory entry structure is illustrated in Listing 4.7.

```
struct directory_entry
{
    unsigned char name[MAX_NAME_LENGTH];
    uint32_t i_node_index;
} typedef directory_entry_t;
```

Listing 4.7: Directory entry structure.

By storing both the entry name and corresponding inode index, directory entries encode the namespace structure of the file system. To simplify entry management and maintain space and lookup efficiency, file and directory names are constrained to a fixed maximum length. Additionally, to ensure all created entries are accessible, names must not be empty or duplicate another entry within the same directory.

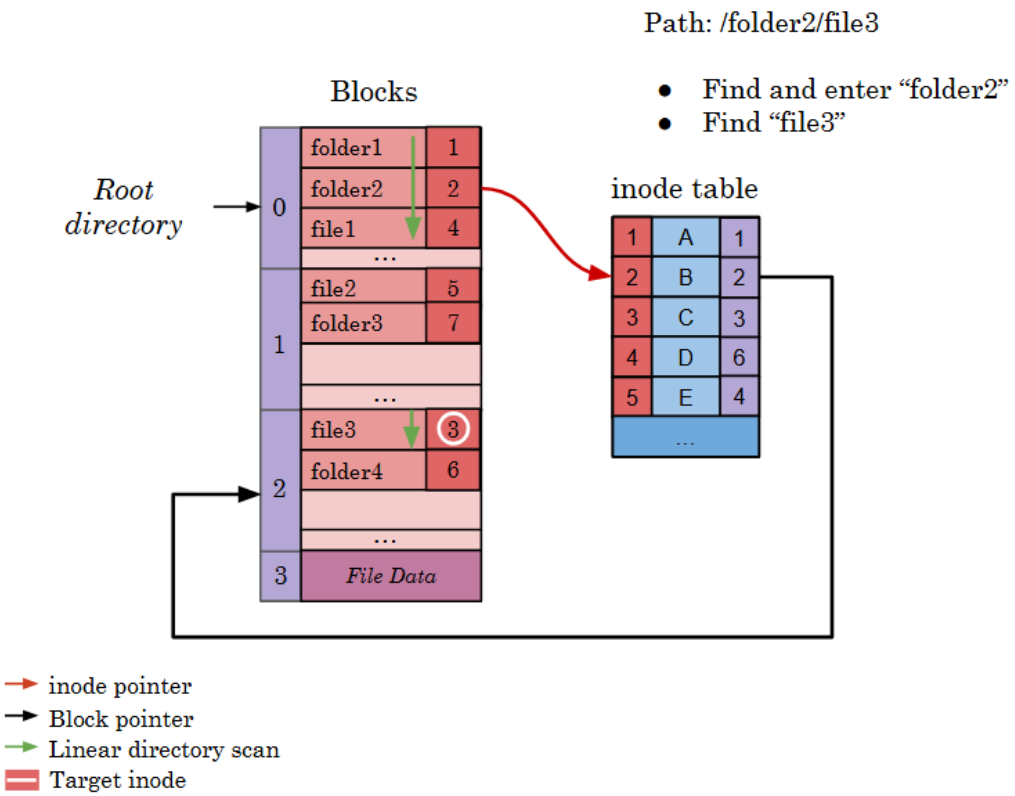


Figure 4.3: Path-based lookup through directory traversal.

This representation enables resolution of entries through path-based lookup, as illustrated in Figure 4.3. A path is represented as a sequence of directory names to traverse, terminating in the target file or directory. Path segments are delineated by the '/' character, with all paths beginning at the root directory, denoted by a leading '/'. The root directory is initialised at system startup as inode 0, with all entries contained beneath it.

Path resolution proceeds through successive traversal of directory entries. For each path segment, the entries of the current directory are searched via a linear scan until a matching name is found. If a match is found and the segment is not the final component of the path, the corresponding inode is accessed and its blocks are searched to continue traversal. However, this continued traversal only occurs if the client has Execute permissions for the directory to be entered, otherwise a permission denied error is returned. If a non-directory entry is encountered before the final segment, the path is deemed invalid. The traversal is implemented recursively, which aligns naturally with the tree structure of the search space.

When the final segment is reached and a matching entry is found, the inode index stored in the corresponding directory entry is returned. If no match is found at any stage, the entry is deemed nonexistent.

File Descriptors

File descriptors are implemented as runtime structures within per-client regions of the file descriptor table (FDT), maintaining open-file state for subsequent file operations. These link a client-accessible handle to a file and its current open state. For minimal abstraction and overhead, the file handles returned to clients are simply the index of the file descriptor within that client's FDT region. A file's open instance may therefore be retrieved by direct indexing into the FDT using the client ID and file descriptor handle.

The Open operation both allocates a given file descriptor and returns the handle to it. This locates a free entry in the FDT through a linear scan of the descriptor entries. To simplify this check, an unused entry is indicated by an inode index value of `UINT32_MAX`, which is otherwise invalid. When a file is closed, the corresponding file descriptor is released by restoring this sentinel value.

File descriptors are realised as structs holding open-state metadata and the associated file inode index, as illustrated in Listing 4.8.

```
struct file_descriptor
{
    uint32_t i_node_index;
    size_t cursor_position;
    file_open_operations_t valid_operations;
    uint8_t padding[3];
} typedef file_descriptor_t;
```

Listing 4.8: File descriptor structure.

By storing the inode index directly, the file server may resolve the associated inode from a file descriptor in constant time through direct indexing into the inode table, and immediately begin file data access or modification. When operations are performed through a file descriptor, the relevant state is supplied automatically where required, for example the cursor position during reads. The valid operations field is represented as an enum value, as shown in Listing 4.10, enabling operation validation while maintaining readability.

```
typedef enum {
    READ_OP = 0b01,
    WRITE_OP = 0b10,
    READ_WRITE_OP = 0b11
} file_open_operations_t;
```

Listing 4.9: File open operations enum.

4.4.2 Operations

These data structures and their associated mechanisms are used to implement the file system operations defined in the design. Each operation is implemented as a function that performs the necessary checks, manipulates the relevant data structures, and returns appropriate results or error codes. The following sections detail the implementation of each operation, highlighting key aspects of their functionality and considerations taken to ensure correctness and efficiency. All operations, excluding Read, Write and Close, operate on paths rather than file descriptors and therefore require path traversal. As this traversal is described fully following Figure 4.3, it is only referenced briefly where relevant.

Namespace Mutation

The organisation of the file system is managed through Create and Delete operations. These are implemented through path validation, traversal and modification of directory, inode and block structures.

Create

A creation operation takes as parameters the path of the entry to create, whether the entry is a file or directory, and the permissions to assign to it. The first step is to locate the inode of the target entry's parent directory, allowing the new entry to be initialised and added as a child of that parent. This ordering allows path validation to occur inherently during traversal, avoiding a separate validation stage and preventing the need to undo partial initialisation should the path prove invalid.

Once the parent directory has been reached, the remaining path consists only of the desired entry name. This name is validated to ensure it falls within the maximum supported length, is non-empty, does not contain the path delineation character, and terminates correctly. The parent directory's blocks are then scanned to ensure no entry with the same name already exists. A further permission check verifies that the calling client holds Write permissions on the parent directory.

Once validation is complete, a free inode is allocated by scanning the inode table for an unset in-use flag, and a starting data block is allocated similarly. If either allocation fails, an appropriate error code is returned. Otherwise, the inode is initialised with the supplied metadata and the allocated block index.

A directory entry is then formed from the entry name and allocated inode index, and inserted into the parent directory. This is achieved by scanning the parent's blocks for free space. If none exists, a new block is allocated, linked to the parent, and used to store the new entry, with the parent inode's size updated accordingly.

Additionally, if creation of a file completes successfully, the library wrapper automatically triggers an Open operation for the issuing client, as the file is likely to be used immediately. Consequently, an opening mode must also be provided during file creation.

Delete

Deletion requires removal of both the target entry and its reference within its parent directory. This requires locating both the target inode and the inode of its parent, achieved through the same traversal mechanism used for lookup, extended to additionally retain the parent inode index. Therefore it takes the target path as the sole operation parameter.

Once the parent inode is located, its blocks are scanned linearly until the matching directory entry is found, which is then marked not in use. Before the target inode may be deallocated, any associated data blocks must also be freed. For files, this involves marking all directly referenced blocks, any indirectly referenced blocks, and the indirect block itself, as free within the BAT.

For directories, deletion is recursive. As simply freeing a non-empty directory's blocks would lose references to nested entries, all contained files and subdirectories must first be deleted. This is achieved by scanning the directory's blocks and invoking the appropriate deletion function recursively for each contained entry. Only once all contents have been removed safely are the directory's own blocks deallocated.

Deleting a file additionally causes it to be closed automatically for the issuing client, avoiding an additional client operation. To preserve multiclient correctness, however, files are only fully deleted once all clients have closed their handles to them. Their directory entry is removed immediately, preventing further opens, and allowing wider directory deletion, but the inode and associated data are retained until no descriptors referencing that inode remain.

This is implemented by scanning the file descriptor table for references to the target inode during deletion. If any remain, a deletion-pending flag is set within the inode rather than deallocating it immediately. Close operations must then check this condition and trigger final deletion once the last handle is released.

Runtime State Management

The management of client specific data is done through Open and Close operations. They are implemented through path validation, traversal and modification of the FDT.

Open

To open a file descriptor for a file, the target inode must first be resolved through the traversal function, with the file path therefore being supplied as a parameter. Alongside this, an opening mode is provided to specify which operations may be performed through the resulting file descriptor. This format is detailed in Listing 4.10.

```
typedef enum {
   READ_OP = 0b01,
   WRITE_OP = 0b10,
   READ_WRITE_OP = 0b11
} file_open_operations_t;
```

Listing 4.10: File opening mode enum.

Once the inode is acquired, the calling client's permissions are validated according to the requested opening mode. If permitted, the client's file descriptor table (FDT) is scanned for a free entry. If one is found, a file descriptor containing the inode index, operation mode and initial file offset of 0 is placed at that location. The index at which this entry is stored is then returned to the client as the handle used to reference the file in subsequent operations.

If no free file descriptor entry is available, an error is returned indicating the client has reached its maximum number of open files.

Close

To close a file descriptor, a client passes the corresponding handle. The Close operation first verifies that this handle is valid, by ensuring it falls within the range 0 to the maximum number of concurrent file descriptors permitted per client. If valid, the corresponding entry in the file descriptor table (FDT) is marked not-in-use.

The operation must also inspect the deletion-pending flag of the inode referenced by that file descriptor. If this flag is set, the system checks whether this was the final open handle to the file by scanning the full FDT for any remaining references to that inode. If none remain, the inode entry is passed to the deletion routine for final removal.

Data Movement

Access and modification of file data are achieved through Read and Write operations. These are realised through coordinated access to file descriptors, inodes and storage blocks. Rather than operating on paths, both operations require the target file to already be open and referenced through a file descriptor handle.

Read

Alongside the target handle, the Read operation additionally takes the requested number of bytes to be read. If this exceeds the client's buffer size, the request is limited accordingly and the client notified through the return code.

If the file descriptor is valid, its opening mode is checked to ensure the Read bit is set. The associated inode is then resolved through the inode index stored in the file descriptor, and the current file offset is used to determine both the block at which reading begins and the offset within that block. This is calculated through integer division and remainder operations on the block size.

The operation then reads data from the resolved block, either through direct or indirect references as required, copying data into the client buffer until either the requested number of bytes has been read or the end of file is reached. If the end of a block is encountered first, the next referenced block in the inode is resolved and reading continues. Reaching the end of the file before satisfying the full request is returned as a partial success.

Once complete, the file descriptor's offset is updated to reflect the new cursor position. This updated offset, together with the number of bytes read, is returned to the client alongside the return code.

Write

The Write operation similarly takes a file descriptor handle and the number of bytes to write, validating both similarly to Read. A buffer containing the data to be written is also required, though this may be supplied either implicitly or explicitly depending on whether the synchronous or asynchronous interface is used.

As with the Read operation, the starting block and offset within that block are calculated. Data from the client buffer is then copied into the file from that position onwards. If the end of a block is reached, writing continues into the next referenced block where one already exists.

If writing reaches the end of the file and additional storage is required, further blocks are allocated, linked into the inode and used to continue the write. Once complete, the inode size metadata and file descriptor offset are updated, and the new offset and number of bytes written are returned to the client. A partial write occurs only if additional blocks are required but no further blocks are available for allocation.

Metadata Access and Modification

Metadata access and modification operations are implemented through traversal and inspection of inode and directory structures, with updates applied where permitted.

Get Size and Get Permissions

These operations each take a single target path as a parameter and utilise the traversal function to resolve the target inode, returning an appropriate error code if this fails. Once the inode is acquired, Read permission is validated for the calling client and, if successful, the appropriate metadata field is extracted from the inode and returned.

Set Permissions

This operation follows the same approach, but additionally takes a new permission code to apply, represented as a bitmask constructed from the enum values shown in Listing 4.11. This allows permissions to be represented compactly within the inode, while still supporting efficient validation through bitwise operations.

```
typedef enum {
   PERM_PRIVATE = 0b000,
   PERM_READ = 0b001,
   PERM_WRITE = 0b010,
   PERM_EXECUTE = 0b100,
   PERM_PUBLIC = 0b111
} permissions_t;
```

Listing 4.11: Permissions bitmask enum.

The permissions field in the inode is updated to this value only if the calling client ID matches the ID stored in the owner field; otherwise, a permissions error is returned.

List

The List operation similarly acquires the inode of the target path through traversal, but result generation is more involved than reading a single inode field. It first verifies that the target inode represents a directory, failing otherwise, and checks that the calling client holds the required Execute permission. It then iterates through the directory's blocks, appending the path of each in-use directory entry to a result buffer, halting addition if continuing would exceed the client's buffer capacity.

Exists

This operation takes a single target path as a parameter and uses the traversal function to search for it. Its implementation differs only in its handling of lookup failure: rather than returning an error when the target is not found, it returns false. If the target is successfully resolved, it returns true.

4.5 Error Handling

The primary sources of errors in this system arise from invalid client requests and resource limitations. These include cases such as invalid permissions, malformed paths or file descriptors, as well as exhaustion of resources such as available blocks or file descriptors. To maintain correctness, the appropriate conditions are explicitly checked before the execution of each stage of the operation.

To support this, operations are decomposed into smaller functions, each responsible for a specific task. At each potential point of failure, inputs and system state are validated, with failures resulting in the return of a defined error code from a global set. These error codes are propagated through the call chain, allowing higher-level operations to respond appropriately. Where possible, alternative actions are attempted; otherwise, the operation is safely terminated, reverting any partial changes to maintain consistency. The final error state is then communicated back to the client through the response structure. Some valid, yet undesirable, conditions such as trying to read a size larger than the buffer size, still cause the client to be notified that the operation was not fully successful.

5 Evaluation

To address both correctness and performance, the entire file server implementation was evaluated via multiple stages of testing and performance analysis.

5.1 Validation

Extensive validation was conducted to evaluate the correctness, robustness and reliability of the file server under both normal and adverse conditions. Given the range of supported operations, numerous edge cases and error scenarios arise. Additionally, the presence of multiple clients and asynchronous communication introduces further complexity, requiring testing not only to verify functional correctness but also to ensure that the system maintains consistency and stability under interleaved workloads.

To increase confidence in the correctness of the system as a whole, testing was conducted at multiple levels. Initially, unit testing was used to verify the functionality of individual components in isolation. This enabled a modular development approach, allowing components to be validated incrementally during implementation. Testing at this level also simplified debugging, as errors were easier to localise due to their limited scope and the reduced code surface.

However, as the system relies heavily on interactions between components and protection domains, integration testing was also necessary. This enabled validation of end-to-end behaviour, ensuring that all operations functioned correctly when executed through the full client–server interface under a range of conditions.

5.1.1 Unit Tests

Unit tests were developed for the core of the file system, including components such as block, inode and directory management and file handle operations. These tests focused solely on verifying the correctness of the file server's data structures and their accesses.

To ensure this isolation, these components were tested by directly invoking their functions outside the Microkit environment. This allowed inputs and supporting data to be simulated through deterministic flows, and the internal state of the file system to be inspected, enabling correctness to be validated at each stage of execution. Testing regular inputs alongside failure conditions, such as resource exhaustion, gave confidence that the fundamental building blocks of the file system operated correctly before integration and would not cause wider issues upon failure.

For example, one test constructs a mock 'indirect block' containing a list of allocated block indices, invokes the `release_indirect_block` routine, and then verifies that each referenced block is marked free in the block allocation table.

5.1.2 Integration Tests

Once the components were verified, integration testing was used to validate the behaviour of the system as a whole, exercising all file system operations through its regular running microkit environment. Rather than directly inspecting internal state, validation was performed through the responses and error codes returned to clients, such as creating a file then listing the root directory. This ensured that testing reflected real system usage and confirmed that correct behaviour was exposed through the public interface.

Single Client

These tests exercised the majority of functionality from the client's perspective, covering the full lifecycle of files, including creation, reading, writing and deletion, without the added complexity of interleaved access. Within this scope, it was possible to verify that invalid client inputs were correctly rejected with appropriate error codes, for example when attempting to delete a non-existent file. Additionally, boundary conditions were tested to ensure correct behaviour under extreme cases, such as reading the maximum supported amount of data.

Multiple Clients

To ensure that all clients were serviced correctly and fairly, and that the system operated reliably under load, extended tests were conducted which interleaved operations from multiple clients. This enabled validation of scenarios that cannot be observed in a single-client context, including permission enforcement and modification of shared resources.

As clients execute independently, explicit synchronisation was required. This was achieved through the use of marker files, where clients created files with predefined names that were monitored by other clients. These acted as barriers, allowing coordination of execution order and enabling clients to signal completion of operations to both the server and one another. For example, one test creates a publicly readable file as client 1, after which client 2 attempts to read that file to verify access is permitted. The file is then modified by client 1 to be private, and client 2 subsequently attempts to read it again, verifying that the operation is correctly rejected with an `INVALID_PERMISSIONS` error.

5.2 Performance Evaluation

5.2.1 Goals

The primary goal of this evaluation was to compare the performance characteristics of the synchronous and asynchronous file server designs. In particular, the evaluation aimed to quantify the impact of IPC overhead and assess the effectiveness of batching in reducing per-operation cost. Additionally, the behaviour of both systems under multiclient workloads was examined to determine how well each design scales under contention.

5.2.2 Metrics

For the per-operation benchmarking, performance was evaluated using the average time per-operation, calculated by measuring the total execution time of a long sequence of the specified operation, to amortise any stochastic effects, and dividing by the number of operations performed. For the more realistic workflows, time taken to complete the whole suite was used. These provided consistent bases for comparing synchronous and asynchronous implementations.

Time measurement was performed using the CPU cycle counter, as the low-level execution environment did not provide native timing facilities. The cycle count was recorded before and after each client's measurable operations, and converted to elapsed time using the processor's clock frequency. Results were averaged across multiple runs to mitigate run-to-run noise and improve reliability.

In multiclient scenarios, fairness was also considered. This was validated by recording the total time taken by each client to complete its workload, ensuring that no individual client experienced disproportionate delay.

5.2.3 Experimental Setup

All experiments were conducted within the same emulated environment using QEMU's default `virt` platform, ensuring consistency across all tests. The synchronous and asynchronous implementations were evaluated under identical conditions, with the same workloads, file sizes and file system state.

For each experiment, the only variables altered were the server design (synchronous or asynchronous), the batching configuration and the number of clients. All operations were validated for correctness during execution to ensure that performance results reflected valid system behaviour rather than erroneous outcomes. During the varied workload, this processing also mimicked non-file I/O that would occur in a typical system, helping to ensure that the measured performance reflected realistic operating conditions.

5.2.4 Benchmark Design

Two benchmark suites were developed to evaluate system performance. The first was designed to isolate and quantify the reduction in communication overhead achieved by the asynchronous server, while the second evaluated overall system performance under more realistic and varied workloads.

This initial aim was achieved through a single-client benchmark which repeatedly executed a single type of operation allowing controlled measurements of individual operation costs for both systems. Both read and write operations were evaluated independently, as they are representative of common file system usage and exhibit differing behaviour. As timing was recorded from the client's perspective, this included both internal server processing time and communication overhead.

For the asynchronous implementation, the benchmark was executed with varying batch sizes. A batch size of one provided a baseline comparison, to ensure that the asynchronous design does not significantly degrade performance when batching is not possible, which is important as many real workloads contain inherent dependencies. Larger batch sizes allowed evaluation of how batching reduces per-operation cost, identifying the point at which performance improvements are significant and where further increases yield diminishing returns.

The second benchmark was a multiclient, realistic workload designed to simulate typical file system usage. Multiple clients executed independently, each performing a sequence of varied operations across many files. This sequence consisted of Create, Write, Close, Open, Seek, Read and Delete operations, representing a complete file lifecycle. Operations were interleaved across clients to introduce contention and better reflect real-world usage patterns.

As batching is central to the performance benefits of the asynchronous design, this benchmark was structured to allow batching where it would naturally occur, without artificially constraining the workload. While it is possible to construct workloads with strict dependencies between operations, this would reduce asynchronous behaviour to that of the synchronous system and provide limited insight beyond the single-operation benchmark.

Instead, operations were grouped into realistic batches:

- **Batch 1:** Create.
- **Batch 2:** Write and Close.
- **Batch 3:** Open.
- **Batch 4:** Seek and Read and Delete.

Performance was measured across varying numbers of clients, enabling analysis of scalability, contention and the impact of interleaved access. This approach provided a balanced evaluation, allowing direct comparison of synchronous and asynchronous designs under both controlled and realistic conditions.

5.2.5 Results and Analysis

Single-Client Per-Operation Benchmark

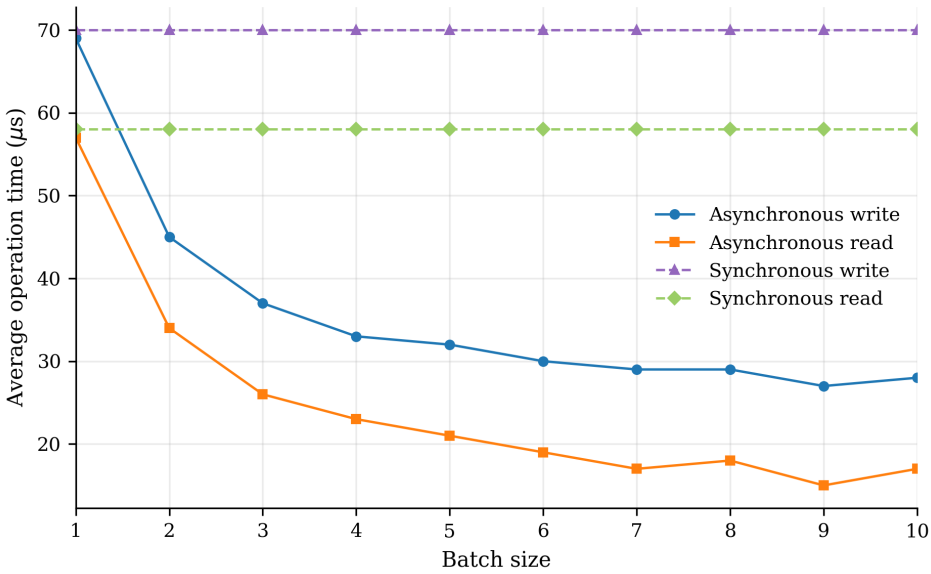


Figure 5.1: Average time per operation by batch size (Read and Write)

Figure 5.1 shows the average time taken per operation against batch size for both read and write operations, alongside the synchronous baseline for comparison. The synchronous system achieves 70 μs for writes and 58 μs for reads, reflecting the lower internal processing cost of read operations in this in-memory system.

When the asynchronous system is configured with a batch size of one, performance is effectively identical to the synchronous case, with writes at 69 μs and reads at 57 μs. This confirms that the asynchronous interface introduces negligible overhead in the absence of batching, validating that it can be used without performance degradation when batching is not possible.

As batch size increases, a substantial reduction in per-operation cost is observed, indicating that communication overhead forms a significant component of total operation latency. For write operations, latency decreases from 69 μs at batch size one to a minimum of 27 μs at batch sizes of seven and above, corresponding to an approximate 60% reduction. Read operations exhibit an even greater improvement, decreasing from 57 μs to approximately 17 μs, yielding reductions of around 70%. The total time reduction per operation, however, is roughly equal for both operations ($\approx 40\ \mu s$), indicating that the relative improvement is greater for reads due to their lower internal processing cost.

The majority of this improvement occurs at relatively small batch sizes, with both the write and read operations exhibiting extremely similar trends, showing this behaviour is consistent over varied conditions. Increasing the batch size from one to four reduces write latency from 69 μs to 33 μs ($\sim$ 52%) and read latency from 57 μs to 23 μs ($\sim$ 60%). Beyond this point, performance improvements become incremental, plateauing completely beyond a batch size of 7, as the system approaches its performance bound.

This trend is further supported by Figure 5.2, which shows the gradient of the per-operation time with respect to batch size. The gradient magnitude decreases rapidly as batch size increases, approaching zero at larger batch sizes.

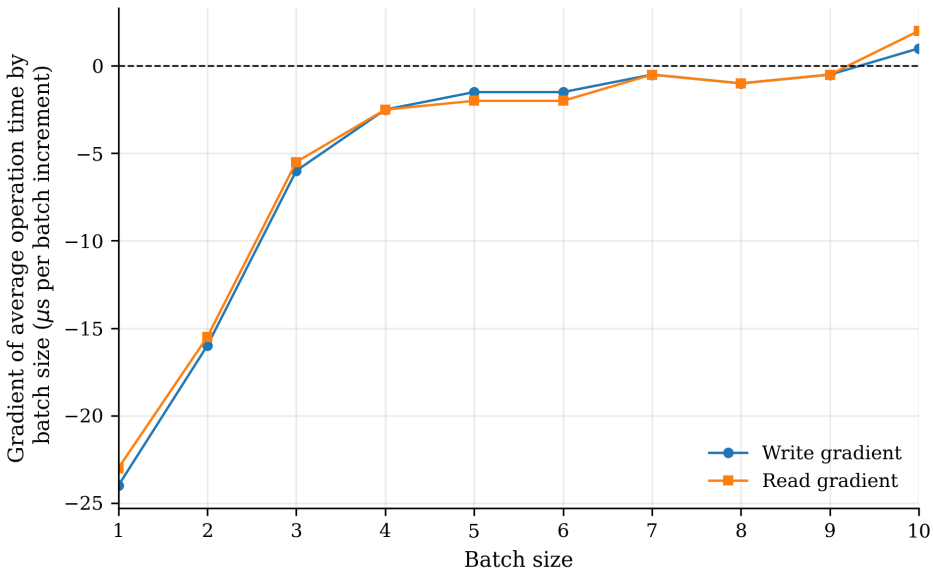


Figure 5.2: Gradient of average time per operation by batch size (Read and Write)

These results closely follow the analytical model discussed in section 3.2.2. The stabilising of performance at larger batch sizes indicates that the communication overhead has been effectively amortised, and the system is now operating close to its computational performance bound. At a batch size of 7, this upper bound is relatively achievable, indicating that IPC overhead could be effectively mitigated in realistic system conditions.

Realistic Multiclient Workload

Figure 5.3 shows the mean execution time per client as a function of the number of concurrent clients for both the synchronous and asynchronous systems. Across all client counts, both systems exhibit relatively stable performance, with the asynchronous design consistently outperforming the synchronous baseline.

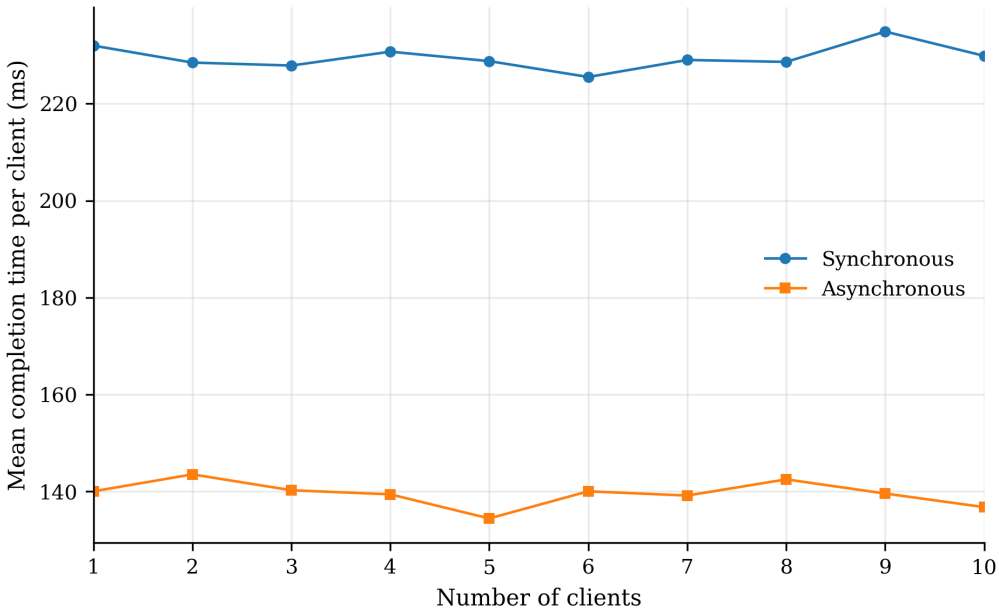


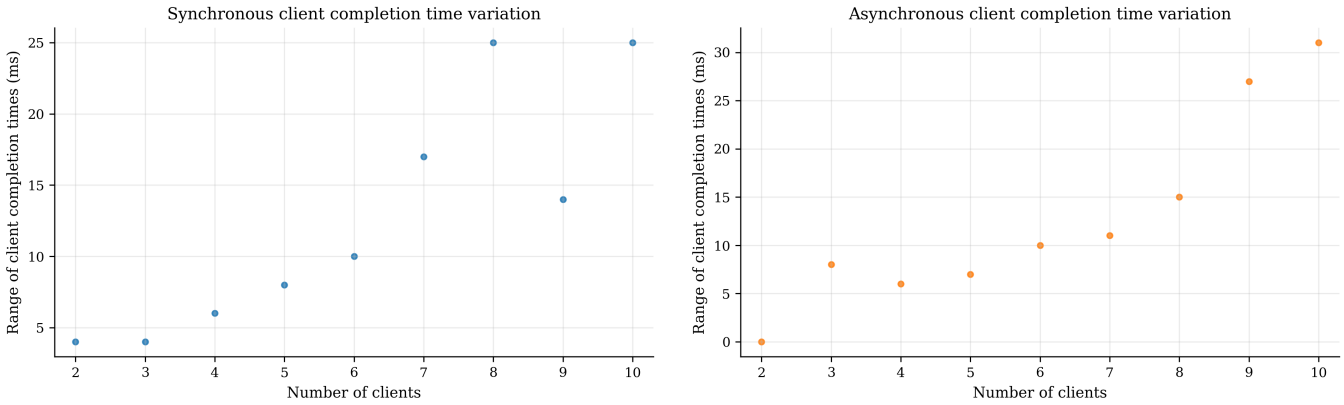
Figure 5.3: Total execution time per client by number of clients (Synchronous vs Asynchronous)

For a single client, the synchronous system took approximately 232 ms to complete the suite, compared to 140 ms for the asynchronous system, representing a reduction of approximately 40%. This improvement remains consistent as the number of clients increases. Expectedly, this reduction is lower than the single-operation benchmark, as there was less opportunity for batching due to dependencies between operations, and the presence of other non-file I/O processing reduced the proportion of time spent on IPC. However, the still substantial improvements across all client counts, confirm that the benefits of reduced communication overhead are maintained at high levels of contentention and in more complex and realistic workloads.

Although there is some variation across client counts, it is minimal and exhibits no clear trend, suggesting that the workload scales predictably and that the systems remain robust as load increases.

Fairness between clients is illustrated in Figure 5.4, which shows the distribution of variation of completion times for individual clients in the synchronous and asynchronous systems respectively. Because the measurement is relatively coarse, capturing only the global time from a client starting to a client finishing rather than its time spent actively processing, it is of limited significance. In particular, it does not reflect fairness during execution, having no periodic measurements, as these are difficult to implement reliably in a single-core pre-emptive system, where execution order depends on scheduler behaviour. However it still provides some safety checks that no client is being excessively delayed.

Both graphs exhibit very similar trends, with the range of completion times remaining relatively low across all client counts. For the synchronous system, the variation increases gradually from 4 ms at 2 clients to 28 ms at 10 clients. A similar pattern is observed for the asynchronous system, where the range grows from 0 ms at 2 clients to 31 ms at 10 clients.



(a) Synchronous

(b) Asynchronous

Figure 5.4: Client completion time variability (min - max) by number of clients

There is a slight upward trend for both systems, which is expected, because, as the number of clients increases, there is greater opportunity for minor scheduling differences, resource contention and execution overlap to accumulate.

While the synchronous system exhibits slightly lower variation, the difference is minimal and does not indicate a significant fairness issue in either system. Both systems maintain relatively consistent performance across clients, suggesting that neither design introduces disproportionate delays for any individual client, even under high contention.

Overall, these results show that the asynchronous design provides substantially better performance under realistic multiclient workloads, while maintaining fairness comparable to the synchronous system. Although the performance gains are lower than those observed in the single-operation benchmark, due to the presence of dependencies and less consistent batching opportunities, the asynchronous system still achieves significant improvements. This confirms that batching and reduced IPC frequency remain effective in more complex and realistic usage scenarios.

6 Conclusion

This dissertation set out to investigate whether interprocess communication overhead in seL4 based systems could be mitigated through architectural and interface-level design choices, specifically via an asynchronous, multiclient file server. This encompassed the design and implementation of a complete system, including not only the file server itself but also the surrounding infrastructure required to support client interaction, memory sharing and communication within the Microkit framework. By reducing the number of synchronous interactions and enabling batching and decoupling of requests, the work demonstrates that meaningful performance improvements are achievable even within the constraints of a formally verified microkernel environment.

The results show that IPC overhead is not merely a function of the underlying kernel primitives, but is significantly influenced by higher-level system design decisions, particularly how clients interact with services, how requests are scheduled and processed and how system components are structured to minimise unnecessary communication. Furthermore, the development of a full, working system highlights that such optimisations must be considered holistically, as performance gains emerge from the interaction between interface design, scheduling behaviour and data management rather than from any single component in isolation.

6.1 Contributions

6.1.1 System Design and Implementation

This work presents a complete user-space file server built using the seL4 Microkit framework, designed with both synchronous and asynchronous interfaces. The system supports multiple concurrent clients, a structured file system abstraction and a thorough set of operations including file data, directory, lifecycle and metadata management.

Key contributions include:

- A modular file system architecture, that can scale and adapt flexibly through its incorporation of block management, inodes and file descriptors, suitable for microkernel deployment.
- A multiclient design supporting permission enforcement and organised client state management.
- Dual interface implementations: a traditional synchronous IPC model and an asynchronous model based on notifications, queues and shared buffers.
- A flexible API layer enabling transparent, simple use of either communication paradigm.
- A full implementation within Microkit, demonstrating practical use of channels, notifications, scheduling and memory management.

The implementation validates that complex, stateful services such as file systems can be realised entirely in user space within seL4, while maintaining structured interaction patterns and correctness guarantees.

6.1.2 IPC Overhead Mitigation Insights

The primary analytical contribution lies in demonstrating how asynchronous communication reduces IPC overhead through batching and reduced synchronisation frequency.

The evaluation yields several key insights:

- Baseline parity: asynchronous communication with batch size 1 performs similarly to synchronous IPC, confirming minimal overhead from the abstraction itself.
- Workload sensitivity: this also means, however, that the benefits of asynchronous IPC are highly dependent on workload characteristics, especially the degree of operation parallelism and 'batchability'.
- Batching effectiveness: increasing batch sizes significantly reduces per-operation latency, demonstrating amortisation of IPC costs.
- Low batch size required for performance maximisation: performance improvements plateau beyond moderate batch sizes, indicating practical upper bounds for batching efficiency.
- Multiclient scalability: in realistic multiclient scenarios, asynchronous design also substantially reduces average operation latency compared to synchronous approaches.

These findings reinforce that IPC overhead mitigation is best addressed at the system design level rather than solely through kernel optimisation[**?**].

6.2 Drawbacks and Limitations

Despite achieving its objectives well, the system has several limitations that constrain both performance and general applicability.

6.2.1 File System Limitations

The implemented file system is functionally rich and reasonably well validated, but could be improved in multiple areas. Additional operations such as file renaming and copying could be introduced to improve usability. Furthermore, lookup performance is suboptimal due to linear search mechanisms; the absence of more efficient indexing structures (e.g. hash tables or trees) contributes to increased latency as the file set grows, yet the current implementation is sufficient for general use and for the experiments undertaken in this report.

6.2.2 Benchmarking Constraints

Evaluation required the development of custom benchmarks due to the specialised environment, low-level platform and non-standard interface. While these benchmarks effectively demonstrate relative performance trends, they limit direct comparability with existing systems. Consequently, results should be interpreted as indicative rather than absolute.

6.2.3 Lack of Persistent Storage

The system operates entirely in memory, with no backing disk. Incorporating persistent storage would introduce additional complexity, particularly around buffering strategies, write-back policies and cache management. This would likely require careful coordination between the storage and IPC layers, representing a substantial extension beyond the current scope. As such, the absence of disk I/O results in highly deterministic and relatively low operation latencies. In a realistic system, with persistent storage, operation timings would be significantly longer and less deterministic due to device latency, queuing effects and variability in access times. As a result, the relative impact of IPC overhead would be reduced, since communication costs would form a smaller proportion of total operation latency. Consequently, while the results clearly demonstrate IPC-related performance trends, they likely represent an upper bound on the benefits of communication overhead reduction in real-world storage-backed systems.

However, such systems would typically operate on multi-core hardware, unlike the single-threaded environment used in this work. In these settings, an asynchronous design such as the one presented here, would considerably offset these reduced benefits by enabling significantly higher throughput through non-blocking execution.

6.2.4 Single-Threaded Execution

The clients and server operate on a single thread, restricting the ability for concurrency to be exploited. This inherently limits throughput and reduces the potential benefits of the asynchronous design. In a multi-core system, multiple clients could concurrently issue requests and process responses in parallel with the server executing operations, significantly increasing overall system utilisation. In contrast, the synchronous model effectively serialises access, as clients must block and wait while the server processes each request, limiting concurrency at the system level.

The architecture presented here is designed to be transferable to a multicore environment with only minor modifications, once such support is readily available within seL4.

This limitation also impacts the interpretation of benchmark results. The observed performance improvements arise primarily from reduced communication overhead through asynchronous batching, rather than from true parallel execution. As a result, the measured gains likely represent a conservative estimate of the performance improvements achievable in a fully concurrent system, where additional benefits from overlapping request generation, processing and response handling would be realised.

6.2.5 Dependency on Workload Characteristics

The effectiveness of the asynchronous model is strongly workload-dependent. In scenarios with low concurrency or minimal opportunity for batching, the advantages over synchronous IPC diminish. This indicates that asynchronous designs are not always superior, but instead most effective in systems with naturally parallel or high-throughput, low-dependency workloads.

6.2.6 Additional Client Burden

The asynchronous design introduces additional complexity on the client side, primarily due to the need to manage request queues and response buffers. Each client must maintain its own data structures to support batching, resulting in an increase in per-client memory usage compared to the synchronous model, where requests are issued and resolved immediately hence requiring only a single buffer and no operation queue.

Although a supporting library was implemented to abstract away much of this functionality, the responsibility for correct and efficient usage still lies with the client. In particular, clients must determine how to group operations into batches, which is inherently workload-dependent and not always straightforward.

Additionally, clients must decide when to flush queued requests and when to synchronise to collect results. Delaying synchronisation increases batching efficiency but introduces latency in observing results, while frequent synchronisation reduces batching opportunities and reintroduces IPC overhead. This creates a trade-off between throughput and responsiveness that must be carefully managed.

Overall, while the asynchronous model provides clear performance advantages, it shifts part of the system complexity from the server to the clients. This increases the burden on client implementation and may make the system harder to use correctly, particularly for developers unfamiliar with asynchronous design patterns or the underlying IPC mechanisms.

6.3 Future Work

Future work should focus on extending the system to a multicore environment, when available, to fully realise concurrency benefits. Additionally, enhancing the file system to support a broader set of operations and more efficient data structures, such as tree-based indexing, would improve scalability, and introducing a caching layer and persistent storage would enable more realistic deployment scenarios. Finally, further exploration of IPC mechanisms, including hybrid synchronous—asynchronous models and adaptive batching strategies, could also yield additional performance improvements.

Bibliography

[1] GeeksforGeeks, "Micro-Kernel and Monolithic Kernel," Accessed: 2026-04-18. [Online]. Available: https://www.geeksforgeeks.org/operating-systems/difference-between-microkernel-and-monolithic-kernel/

[2] Y. Nakata, "ACL vs Capability," Accessed: 2026-04-18. [Online]. Available: https://www.chikuwa.it/blog/2023/capability/

[3] T. seL4 Foundation, "Microkit User Manual (2.2.0) | seL4 docs," Accessed: 2026-04-18. [Online]. Available: https://docs.sel4.systems/projects/microkit/manual/latest/

[4] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts Essentials*, 2nd ed. Wiley Publishing, Oct. 2013.